DOCUMENTAÇÃO DO PROJETO

Spillway, Relay e Forecast

Documentação geral para o dono do projeto

Arquitetura, serviços, fluxos e as decisões por trás de cada escolha.

Versão de 30/10/2026 · Leonardo Matos

Sumário

1. Visão geral

1.1 O que você construiu, em uma página
1.2 O mapa do ecossistema
1.3 Como ler esta documentação
1.4 Os repositórios
1.5 As cinco ideias que explicam o desenho

2. spillway-data: o pipeline de dados

2.1 O que ele faz
2.2 As fontes de dados
2.3 A arquitetura em camadas (bronze, silver, gold)
2.4 O que o DAG faz, passo a passo
2.5 Como o número final é calculado (a regra de negócio)
2.6 O que chega ao banco
2.7 Backfill e reprocessamento
2.8 Onde roda
2.9 As novas fontes do ONS (sistema, água e rede)

3. spillway-api: a API e as regras de negócio

3.1 Papel
3.2 Mapa das rotas
3.3 Camadas da aplicação
3.4 Contas, segurança e acesso
3.4.1 Quem pode ver o quê no site
3.5 Cache: como o site fica rápido sem ficar velho
3.6 Exportações (PDF, Excel, CSV)
3.7 Relatórios e gráficos dentro deles
3.8 E-mails: modelos, variáveis e disparos
3.9 Previsão publicada pelo banco
3.10 Migrações do banco
3.11 Endpoints do sistema elétrico, da água e da rede

4. spillway-web: o site

4.1 Papel e tecnologia
4.2 O mapa do site (menu por pergunta)
4.3 Quem pode ver o quê
4.4 Como as páginas carregam (e por que ficam rápidas)
4.5 Rotas internas (/api/*)
4.6 Pontos de usabilidade que valem registrar

4.7 Variáveis de ambiente principais
4.8 Testes e qualidade

5. spillway-infra: a infraestrutura na AWS

5.1 Papel
5.2 A arquitetura
5.3 Módulos Terraform
5.4 Os usuários do banco e as permissões
5.5 Decisões de infraestrutura
5.6 O que falta para entrar no ar
5.7 Custo mensal estimado

6. Relay: a plataforma de mensagens

6.1 O problema que ele resolve
6.2 A arquitetura
6.3 A vida de uma mensagem
6.4 Sequência de um envio feliz
6.5 Falhas e novas tentativas
6.6 Troca de provedor e circuit breaker
6.7 Limite de taxa para toda a frota
6.8 Idempotência em cada salto
6.9 Ordem por destinatário
6.9.1 Templates e campos dinâmicos
6.10 Provedores reais
6.11 A API do Relay
6.12 Observabilidade
6.13 Números medidos
6.14 Como subir localmente

7. Os satélites do Relay: gateway, recibos e Terraform

7.1 relay-api-gateway: a porta de entrada simples
7.2 relay-receipts: o que o provedor diz depois
7.3 O que o Relay faz com o recibo
7.4 Por que SQS entre as duas Lambdas, e não API Gateway
7.5 Logs estruturados
7.6 relay-terraform: tudo da AWS do Relay em código

8. spillway-forecast: a previsão de risco de corte

8.1 O que ele prevê
8.2 Os dados
8.3 O que o modelo recebe (as variáveis)
8.4 O modelo e por que ele
8.5 As linhas de base (contra o que o modelo é comparado)

8.6 Como se valida (sem se enganar)
8.7 Resultados (teste final, 2026-04 a 2026-09)
8.8 Limitações
8.9 Como a previsão chega ao site
8.9.1 O que a página mostra
8.10 Ciclo automático (em produção local desde 28/10/2026)
8.11 Como rodar

9. Fluxos de ponta a ponta

9.1 Cadastro de um membro (com código por e-mail)
9.2 Um novo mês do ONS: do arquivo ao e-mail
9.3 Ativar a newsletter na home
9.4 Baixar a análise em PDF
9.5 O recibo de entrega de um e-mail
9.6 Da previsão ao site
9.7 Esqueci a senha
9.8 Resumo: onde cada decisão aparece

10. Operação: como subir, onde está cada coisa, o que fazer quando algo quebra

10.1 O mapa de portas (ambiente local)
10.2 Subindo tudo, na ordem
10.3 Dados: backfill e relatórios
10.4 Envios de e-mail
10.5 Testes e verificações por repositório
10.6 Onde ficam segredos e configurações
10.7 Lista de problemas que já aconteceram (e a solução)
10.8 Cuidados antes de alterar algo

11. As decisões de arquitetura, em linguagem simples

11.1 Spillway (ADRs do spillway-api)
11.2 Relay (ADRs do relay)
11.3 Relay-receipts
11.4 Forecast
11.5 Decisões de produto e de experiência
11.6 Novas fontes do ONS

12. Estado atual, pendências e riscos

12.1 O que está pronto e funcionando (no seu computador)
12.2 O que foi concluído por último e o que está aberto
12.3 O que ainda não existe
12.4 Riscos e pontos de atenção
12.5 Próximos passos sugeridos
12.6 Resumo em números

12.7 Novas fontes do ONS (fases 0 a 4)

13. Glossário

Setor elétrico

Dados e engenharia

Relay e mensageria

Previsão e aprendizado de máquina

1. Visão geral

1.1 O que você construiu, em uma página

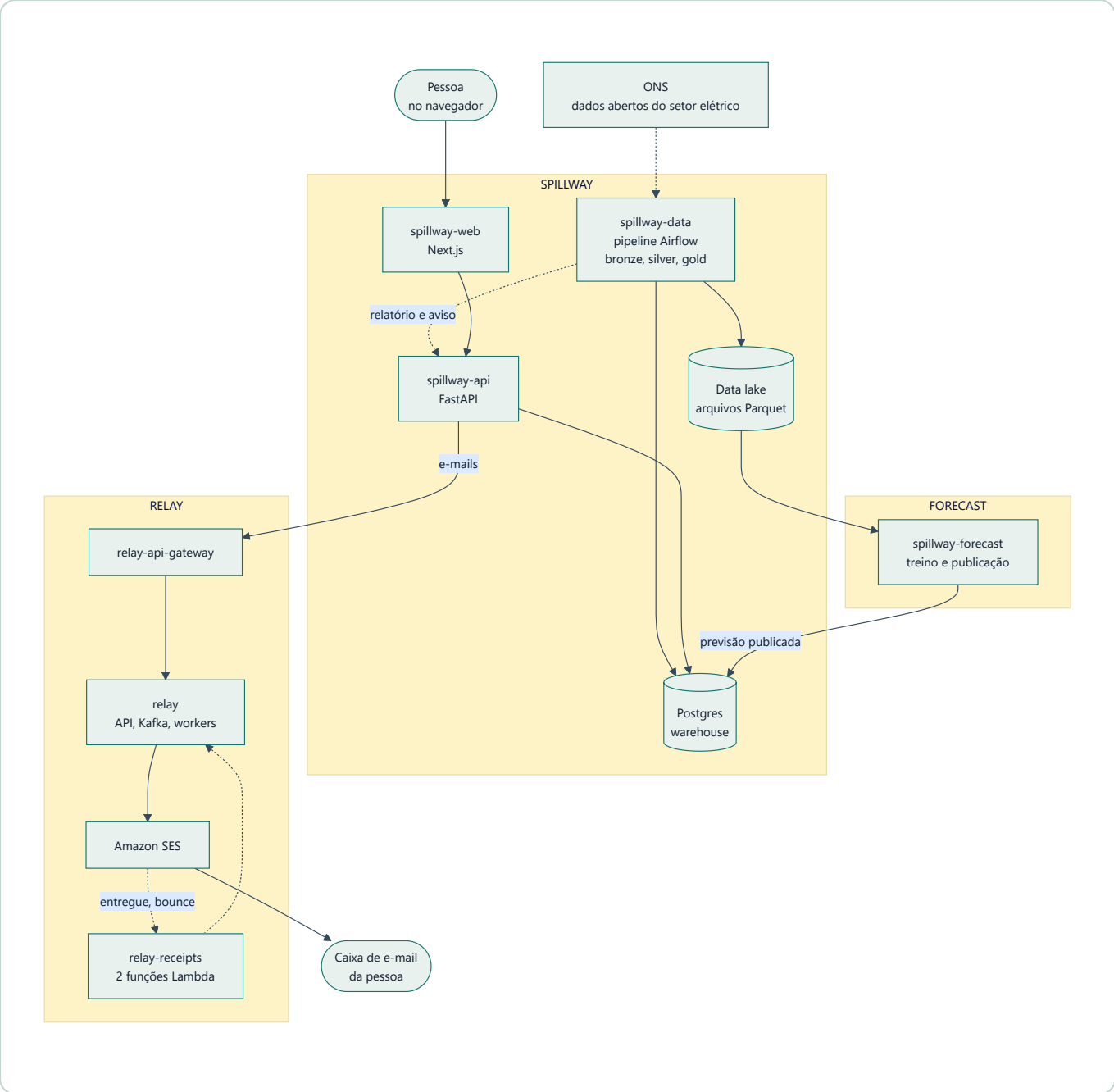
Você tem **três produtos que se encaixam**, cada um com um papel claro:

Produto	Para que serve	Repositórios
Spillway	Mostrar quanta energia eólica e solar o Brasil deixa de gerar (o *curtailment*), mês a mês, estado a estado, usina a usina. É o site, com área de membros, relatórios, análises e previsão.	`spillway-data`, `spillway-api`, `spillway-web`, `spillway-infra`
Relay	Uma plataforma genérica para **enviar mensagens** (e-mail, SMS, WhatsApp, push) com garantias: nada se perde, tenta de novo, troca de fornecedor se um cair. O Spillway usa o Relay para mandar e-mails. Outros produtos futuros poderão usar também.	`relay`, `relay-api-gateway`, `relay-receipts`, `relay-terraform`
Forecast	Um modelo de aprendizado de máquina que estima **o risco de corte de energia hora a hora**, e a rotina que o treina, o publica e o mantém atualizado.	`spillway-forecast`

> **Em resumo:** o Spillway é o produto que as pessoas veem. O Relay é a "central de correios" que o Spillway usa para falar com elas por e-mail. O Forecast é o "cérebro estatístico" que alimenta a página de previsão.

1.2 O mapa do ecossistema

O diagrama mostra quem fala com quem. As setas pontilhadas são caminhos que acontecem em segundo plano (agendados ou assíncronos).



1.3 Como ler esta documentação

- **Capítulos 2 a 5** explicam cada parte do **Spillway** (dados, API, site, infraestrutura).
- **Capítulos 6 e 7** explicam o **Relay** em profundidade, porque é a peça com mais decisões de engenharia.
- **Capítulo 8** explica o **Forecast**: dados, modelo, por que esse modelo, como é validado e como se mantém atualizado.
- **Capítulo 9** junta tudo em **fluxos de ponta a ponta** (diagramas de sequência).
- **Capítulo 10** é o **manual de operação**: como subir tudo, portas, variáveis, problemas comuns.
- **Capítulo 11** reúne as **decisões de arquitetura** em linguagem simples.
- **Capítulo 12** é o **estado atual**: o que está pronto, o que falta e os riscos.

- O **glossário** (capítulo 13) explica os termos técnicos e do setor elétrico.

Em cada capítulo, as caixas coloridas têm um papel fixo:

> **Por quê?** explica o motivo de uma decisão (e o que foi descartado).

> **Atenção:** avisa de um limite, risco ou pendência.

> **Dica:** mostra um atalho ou um jeito de usar.

1.4 Os repositórios

Todos ficam na pasta `projects` e no GitHub do usuário `LeonardoMatos07`. As pastas locais de quatro deles ainda têm o nome antigo (`energia-*`) por causa de uma renomeação que ficou pendente.

Pasta local	Repositório no GitHub	Linguagem e stack	Papel
`energia-data`	`spillway-data`	Python, Airflow, Polars	Pipeline de dados do ONS
`energia-api`	`spillway-api`	Python, FastAPI, SQLAlchemy	API, regras de negócio, e-mails, exportações
`energia-web`	`spillway-web`	TypeScript, Next.js, Recharts	O site
`energia-infra`	`spillway-infra`	Terraform	Infraestrutura na AWS
`relay`	`relay`	TypeScript, NestJS, Kafka, Postgres, Redis	Plataforma de mensagens
`relay-api-gateway`	`relay-api-gateway`	TypeScript, Node puro	Porta de entrada simples para o Relay
`relay-receipts`	`relay-receipts`	Python, AWS Lambda	Recibos de entrega dos provedores
`relay-terraform`	`relay-terraform`	Terraform	SES, filas e Lambdas na AWS
`spillway-forecast`	`spillway-forecast`	Python, scikit-learn, MLflow	Previsão de risco de corte
`spillway-docs`	(local)	Markdown, Mermaid	Esta documentação

> **Atenção:** o contrato entre os quatro repositórios do Spillway está escrito em `spillway-api/docs/phase1-contract.md`. Quando algo compartilhado muda (uma rota, uma tabela, uma variável de ambiente), a mudança começa por esse documento.

1.5 As cinco ideias que explicam o desenho

1. **Dados confiáveis antes de bonitos.** O pipeline guarda o arquivo original, valida, separa linhas ruins e só então publica. O site nunca mostra um número que não passou nas verificações.
2. **A API é a dona do banco.** Quem produz dados (pipeline, forecast) só escreve nas tabelas que a API definiu. Isso evita que dois sistemas discordem sobre o formato.
3. **Nada é enviado de forma síncrona se puder falhar.** E-mails vão para o Relay, que guarda, tenta de novo e troca de fornecedor. O site nunca fica esperando o SES.
4. **Tudo que pode ser repetido é seguro de repetir (idempotência).** Rodar o pipeline duas vezes, reenviar uma requisição, publicar a mesma previsão: o resultado é o mesmo, sem duplicar nada.
5. **Cada serviço pode cair sozinho.** Se o forecast parar, o site mostra a última previsão com a idade dela. Se o Relay parar, a API não perde o e-mail. Se o SES recusar, o Relay tenta outro caminho.

2. spillway-data: o pipeline de dados

2.1 O que ele faz

Todo mês o ONS (Operador Nacional do Sistema Elétrico) publica arquivos abertos com a energia eólica e solar que **deixou de ser gerada** por ordem do operador, em intervalos de 30 minutos, por usina e por conjunto de usinas. O `spillway-data` baixa esses arquivos, valida, transforma, calcula o impacto financeiro e carrega tudo no Postgres, pronto para a API consultar.

Ele roda no **Apache Airflow** (versão 3.3.2), que é o agendador: ele sabe quais passos existem, em que ordem, o que fazer quando um falha e mostra tudo numa interface (porta 8081 localmente).

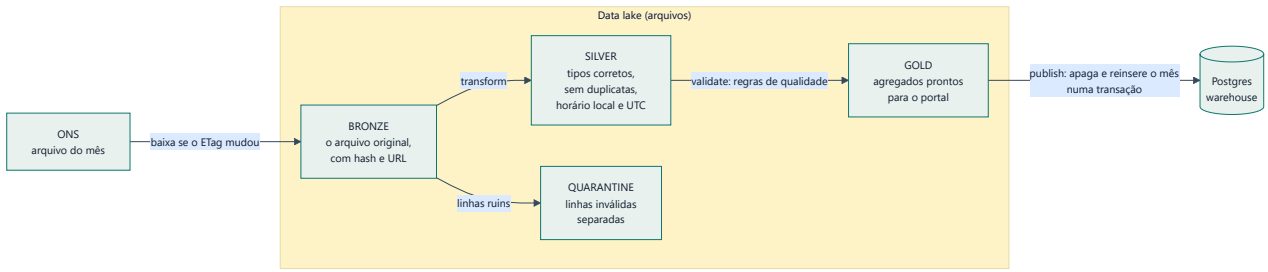
2.2 As fontes de dados

Fonte (ONS)	O que traz
Restrição de operação eólica e fotovoltaica, por **conjunto** (cluster)	O total oficial cortado, a cada 30 minutos. É o número que vale.
Restrição de operação eólica e fotovoltaica, por **usina**	A estimativa por usina e os motivos do corte
Geração por usina	Quanto cada usina realmente gerou
Capacidade instalada	Potência de cada usina (MW) e cadastro
Curva de carga	Consumo do sistema
Custo Marginal de Operação (CMO)	O preço de referência para valorar a energia perdida

Uma segunda DAG (`ons_grid_pipeline`) cuida dos dados da rede elétrica; a principal para o site é `ons_curtailment_pipeline`.

2.3 A arquitetura em camadas (bronze, silver, gold)

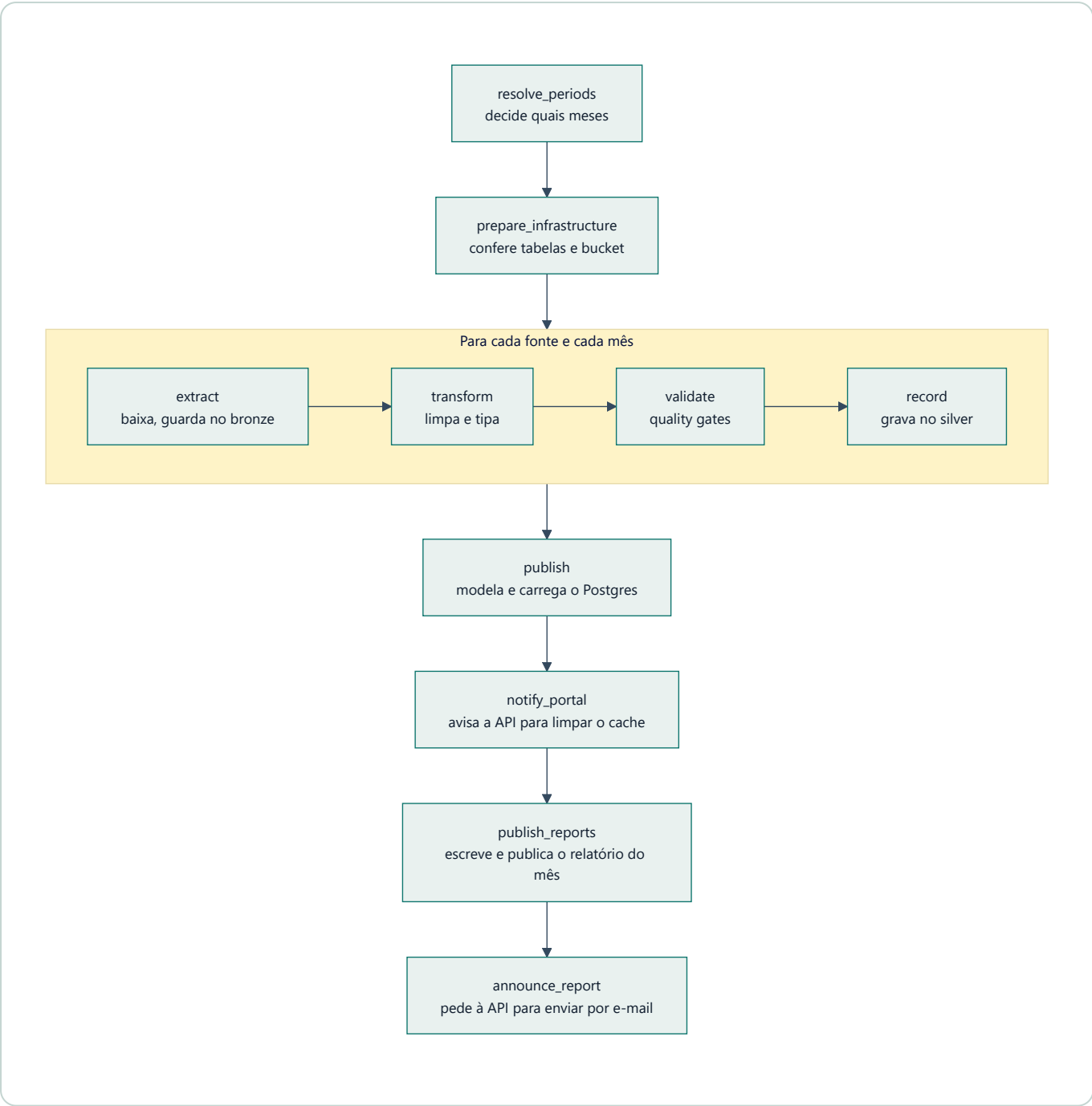
Os dados passam por três estágios. É uma prática comum de engenharia de dados chamada **arquitetura medalhão**:



> **Por quê?** Guardar o **arquivo original (bronze)** permite reprocessar tudo se descobrirmos um erro de regra, sem depender de o ONS ainda ter o arquivo. A **silver** é a fonte confiável (também usada pelo Forecast). A **quarentena** evita que uma linha estranha derrube a carga inteira: ela é separada e contada, e se passar de 1% a carga é bloqueada.

2.4 O que o DAG faz, passo a passo

O DAG principal roda **no dia 3 de cada mês às 06:00** e reprocessa os **últimos 3 meses** (o ONS costuma revisar números já publicados).



Quatro dessas etapas merecem uma explicação:

1. **extract** só baixa se o `ETag` do arquivo mudou. Rodar o DAG de novo sem dado novo não baixa nada.
2. **validate** aplica as regras de qualidade. Qualquer verificação de nível "ERRO" **bloqueia a publicação**.
3. **publish** apaga e reinsere o mês inteiro **numa única transação**. Se algo falhar, o banco continua com o dado anterior.
4. **announce_report** só envia e-mail na **rodada mensal regular** e só para o mês mais recente. Um *backfill* (carga de anos antigos) **nunca** manda e-mail.

> **Atenção:** as duas últimas etapas (`publish_reports` e `announce_report`) foram acrescentadas depois, para fechar o ciclo "dado novo, relatório publicado, inscritos avisados". O DAG foi conferido (importa sem erros e lista as tarefas), mas só dispara de verdade na primeira rodada mensal real.

2.5 Como o número final é calculado (a regra de negócio)

Este é o coração do produto e está documentado como decisão de arquitetura (ADR 2).

1. **O número oficial vence.** O total cortado de cada conjunto é a "geração não realizada apurada" (GNRa): potência média em MW multiplicada por 0,5 hora.
2. **Divide por motivo**, proporcional aos minutos de restrição. Os motivos oficiais do ONS são:
   - **REL**: razão de confiabilidade elétrica (a rede precisava ser protegida);
   - **CNF**: indisponibilidade externa (uma linha ou subestação fora da usina estava desligada);
   - **ENE**: razão energética (havia mais energia do que consumo);
   - **outros**.
3. **Distribui para as usinas**, proporcional à perda estimada de cada uma nas meias horas em que houve restrição.
4. **Valora em reais**: cada meia hora cortada é multiplicada pelo CMO do subsistema, limitado pelo piso e teto anuais do PLD.

> **Por quê?** Estimar por usina sozinho dá números que não fecham com o total oficial. Fazer o oficial mandar e **distribuir** garante que a soma por usina, por estado e por subsistema é sempre igual ao total do ONS. Há uma rotina de **reconciliação** que soma os arquivos originais e compara com o que está no banco (tolerância de 1%).

> **Atenção:** o texto do ADR 2 troca a descrição de REL e CNF (diz que REL é indisponibilidade externa). O site e o ONS usam: REL = confiabilidade elétrica, CNF = indisponibilidade externa. Vale corrigir o texto do ADR.

2.6 O que chega ao banco

O pipeline escreve **apenas** em tabelas que a API já criou. Se faltarem, ele para e pede para rodar as migrações da API.

Tabela	Conteúdo
`plant`	Cadastro das usinas e conjuntos (nome, estado, subsistema, empresa, ponto de conexão)

Tabela	Conteúdo
`plant_monthly_curtailment`	Por usina e mês: geração, corte total e por motivo, perda em R$, rankings
`state_monthly_curtailment`, `subsystem_monthly_curtailment`	Agregados por estado e subsistema (e SIN = Brasil), por fonte
`hourly_curtailment`	Perfil por hora do dia, tipo de dia (útil ou fim de semana) e mês
`load_run`	Histórico de cada execução do pipeline

2.7 Backfill e reprocessamento

Um **backfill** carrega muitos meses de uma vez (foi o que carregou de 2021-10 a 2026-09). Pelo Airflow, dispara-se o DAG com os parâmetros `first_month` e `last_month`.

> **Dica:** o ONS publica a eólica desde 2021-10 e a solar desde 2024-04. Não existem dados anteriores a essas datas nessa fonte.

2.8 Onde roda

Ambiente	Onde	Armazenamento
Local	Docker Compose (Airflow, Postgres do warehouse na porta 5433, SeaweedFS como lake)	SeaweedFS (porta 8333, compatível com S3)
Produção	Uma máquina EC2 sem SSH (acesso por SSM) na AWS	Bucket S3 versionado e criptografado

2.9 As novas fontes do ONS (sistema, água e rede)

Além do curtailment, o `spillway-data` agora carrega mais quatro grupos de dados do ONS. Cada grupo é um **pipeline** (um DAG no Airflow, rodando no dia 3 de cada mês, reprocessando os últimos 3 meses) e escreve em tabelas novas do banco, sempre em transação única e sem duplicar.

Pipeline (DAG)	Fontes	Tabelas no banco
`system` (`ons_system_pipeline`, 07:00)	balanço de energia, carga, geração por usina, capacidade, intercâmbio nacional e internacional, despacho térmico	`subsystem_hourly_balance`, `subsystem_monthly_generation`, `power_plant`, `plant_monthly_generation`, `interchange_hourly`, `interchange_monthly`, `thermal_dispatch_monthly`

Pipeline (DAG)	Fontes	Tabelas no banco
`hydrology` (`ons_hydrology_pipeline`, 08:00)	cadastro de reservatórios, EAR e ENA por subsistema e por reservatório, vazões e cotas, energia vertida turbinável	`reservoir`, `subsystem_storage_daily`, `reservoir_daily`, `plant_monthly_spilled_energy`
`grid_assets` (`ons_grid_assets_pipeline`, 09:00)	linhas de transmissão, subestações, transformadores	`transmission_line`, `substation`, `transformer_capacity` (com histórico)



> **Por quê um pipeline por assunto?** Cada assunto tem seu ritmo, seus arquivos e seus erros. Separar evita que um problema na hidrologia atrase o relatório de curtailment, que já tem e-mail mensal ligado a ele.

O que os dados reais ensinaram (e virou regra no código):

- **Intercâmbio:** o ONS lista cada link na direção do fluxo da hora, sempre com valor positivo. A camada silver fixa a direção de cada link (N→NE, N→SE, NE→SE, S→SE) e dá sinal ao fluxo: positivo é de origem para destino.
- **Balanço:** o arquivo traz um subsistema `SIN` (total); ele é recalculado a partir dos quatro subsistemas para não entrar em dobro. Leituras solares levemente negativas à noite (−0,16 MW) são ruído e são toleradas.
- **Reservatórios:** a energia armazenada de um reservatório é **negativa** abaixo do volume mínimo (2015: até −83%) e passa de 100% quando ele vaza (até 379%). Só valores absurdos (erro de unidade) bloqueiam a carga.
- **Códigos:** o ONS usa três códigos para o mesmo reservatório (planejamento, `res_id` e `cod_usina`); o cadastro faz a ponte e alguns reservatórios sem código ligam pelo nome.
- **Térmicas novas** podem não ter código de planejamento: a chave cai para o CEG.
- **Rede com histórico:** a cada mês o cadastro inteiro é lido e só se escreve uma linha nova quando algo mudou (`valid_from` e `valid_to`), para o mapa poder mostrar a expansão da rede.

> **Atenção:** o backfill da hidrologia começou em **2015** (já carregado localmente até 2026-09); o histórico anterior (EAR e ENA desde 2000, dados hidrológicos desde 1970) é uma execução à parte, bastando recuar `HYDRO_START`.

3. spillway-api: a API e as regras de negócio

3.1 Papel

A `spillway-api` é o **cérebro e o guardião do banco**. Ela:

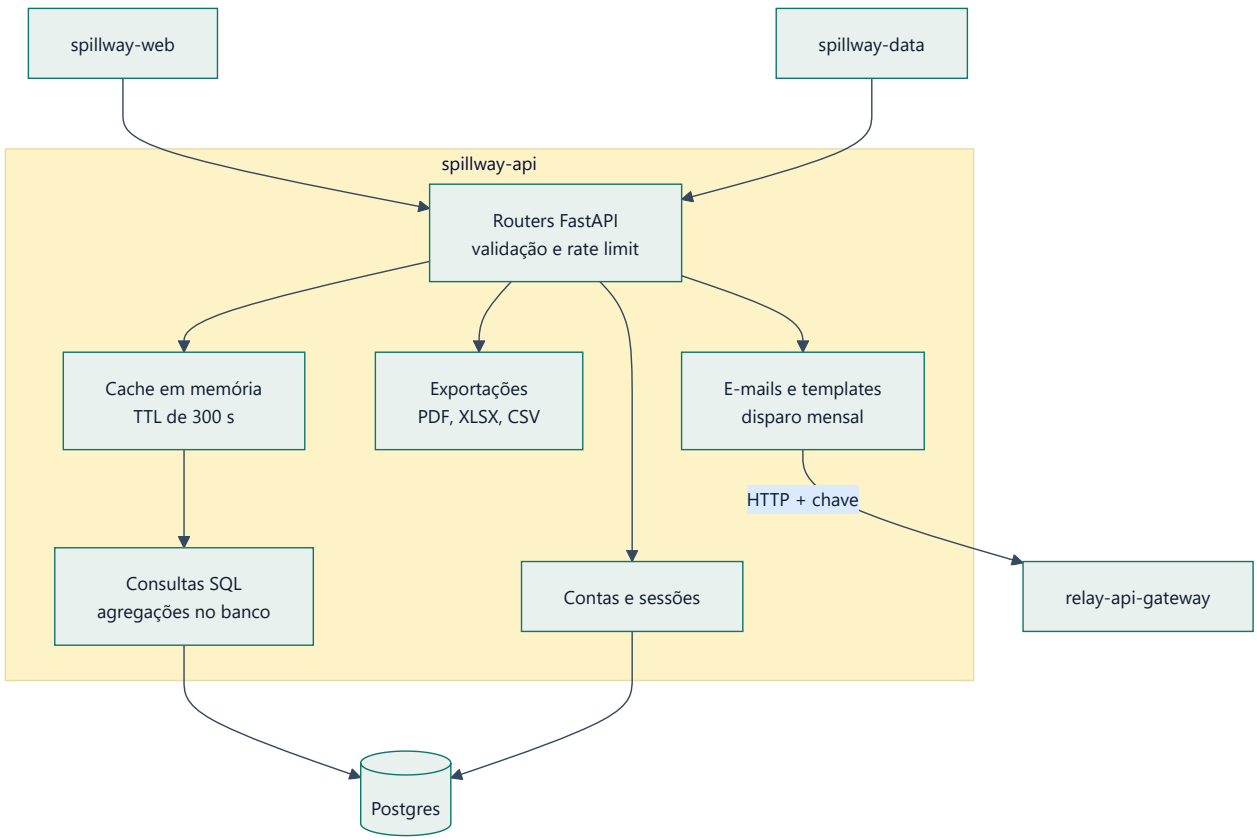
- é a **única dona do esquema do Postgres** (migrações com Alembic, hoje até a `0007`);
- responde ao site com os dados de curtailment, usinas, rankings, relatórios e previsão;
- cuida de **contas de membros** (cadastro com código por e-mail, login, sessão);
- gera **PDF, Excel e CSV** das análises e o PDF dos relatórios;
- monta e **dispara e-mails** (código de verificação, relatório mensal, boas-vindas) pelo Relay;
- registra **eventos de uso** do site para a área administrativa.

Stack: Python 3.12, FastAPI, SQLAlchemy 2, psycopg, Alembic, Pydantic. Testes com pytest (cerca de 240), verificação de tipos com mypy estrito e lint com ruff.

3.2 Mapa das rotas

Grupo	Rotas principais	Quem usa
Dados	`/v1/curtailment/{subsystems, states/{uf}, summary, hourly}`, `/v1/plants`, `/v1/rankings/{operators, connection-points}`	Site (páginas de dados)
Relatórios	`/v1/reports`, `/v1/reports/{slug}`, `/v1/reports/{slug}/pdf`	Site
Exportações	`/v1/exports/analysis.pdf`, `/v1/exports/analysis.xlsx`, `/v1/exports/analysis-csv.zip`	Site (botão Baixar)
Conta	`/v1/auth/{signup, verify-email, resend-verification, login, me, logout, forgot-password, reset-password}` e `/v1/auth/me/newsletter`	Site
Newsletter	`/v1/newsletter` (+ `/confirm`, `/unsubscribe`)	Site
Previsão	`/v1/forecast`, `/v1/forecast/scopes`, `/v1/forecast/model`	Site
Uso	`/v1/events`	Site (rastreio de visitas)
Administração	`/v1/admin/{reports, revalidate, dispatches, leads, usage/*, load-runs}`	Pipeline e administradores

3.3 Camadas da aplicação



3.4 Contas, segurança e acesso

Há dois mundos de autenticação, por motivos diferentes:

Quem	Como autentica	Detalhes
Membros (público cadastrado)	E-mail e senha, com **código de 6 dígitos por e-mail** para verificar o endereço	Senha com `scrypt`. A sessão é um token aleatório cujo **hash** é guardado (o token em si nunca fica no banco). Dura 14 dias. O código vale 30 minutos e tem limite de tentativas.
Administradores e máquinas (pipeline)	**Amazon Cognito** (JWT)	Administradores com MFA obrigatório. O pipeline usa credenciais de máquina (*client credentials*) com o escopo `admin`.

Proteções que valem a pena conhecer:

- **Limite de requisições** por IP em cadastro, login, newsletter e downloads (por exemplo, 5 por minuto na newsletter, 20 por minuto nos downloads).

- **Cadastro com e-mail já existente** responde `409` "já existe uma conta", sem enviar e-mail (decisão sua: a clareza para o usuário vale mais que esconder se o e-mail existe).
- O site repassa o IP real do visitante à API usando um segredo compartilhado (`WEB_API_SECRET`), para o limite por IP funcionar atrás do proxy.

3.4.1 Quem pode ver o quê no site

Só a **página inicial, a metodologia, a privacidade, o cadastro e o login** são abertas. Painel, estados, usinas, relatórios, análises, comparação, previsão e conta exigem login.

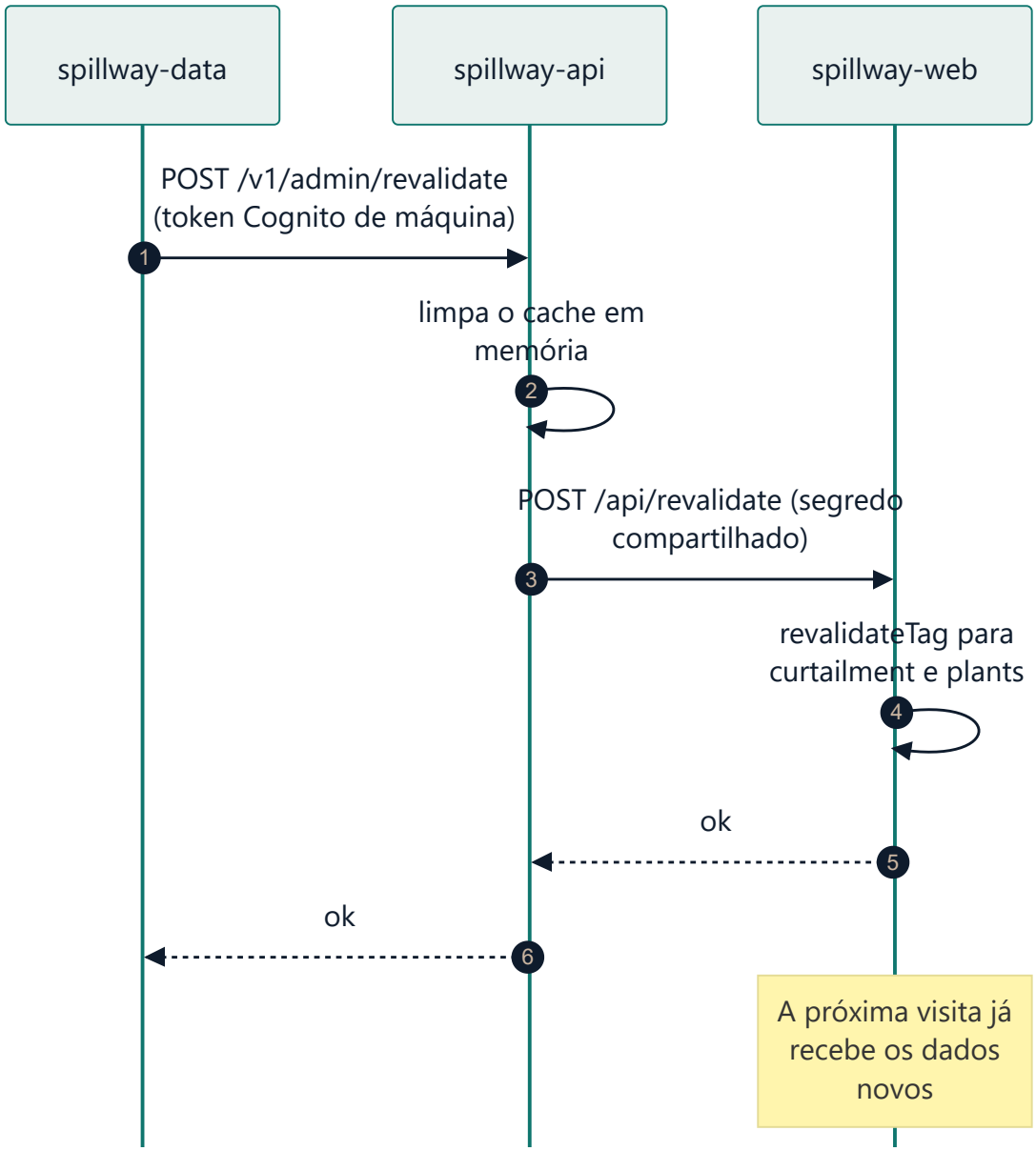
> **Por quê?** O site passou a ser um produto para membros (gera relacionamento, base para o e-mail mensal). Os dados da API continuam técnicos e abertos; a regra de quem vê o quê é aplicada na camada do site (middleware mais verificação da sessão no layout).

3.5 Cache: como o site fica rápido sem ficar velho

Há **três camadas de cache**, cada uma com um motivo:

Camada	Duração	Quem limpa
Cache em memória da API	300 s	Rota administrativa `revalidate`
Cache de `fetch` do Next.js no site	1 hora (30 minutos na previsão)	A mesma rota, por **tags** (`curtailment`, `plants`, `reports`)
Navegador (roteador do Next)	30 s nas páginas dinâmicas	Expira sozinho

Quando o pipeline termina uma carga, ele não espera as horas passarem: avisa a API, que limpa o cache dela e chama o site para invalidar as tags (decisão 4, no capítulo 11).



3.6 Exportações (PDF, Excel, CSV)

Na página de Análises, o botão **Baixar** tem duas opções:

- **PDF**: segue as três visões da tela (Compensação, Empresas e conexões, Perfil horário) com **todas** as linhas e um gráfico horário. Feito com `reportlab`.
- **Dados completos**: um **Excel** com uma aba por tabela (resumo, subsistemas por mês, estados por mês, empresas, pontos de conexão, perfil horário e **todas as usinas por mês**), ou um **ZIP de CSVs** com o mesmo conteúdo.

O PDF e os dados respeitam o **filtro da tela** (escopo, fonte, período): se você escolhe um mês, sai só aquele mês.

> **Por quê?** Um CSV só guarda uma tabela. Para entregar "todos os dados" em um único arquivo, o Excel é o formato natural. Para quem quer CSV, o ZIP mantém um arquivo por tabela.

3.7 Relatórios e gráficos dentro deles

Cada relatório mensal é um texto em **Markdown** gerado pelo pipeline. Para ter gráficos, o Markdown aceita um bloco especial:

````
```chart
{"type":"bar","title":"...","unit":"GWh","labels":[...],"series":[{"name":"...","values":[...]}]}
```
````

O mesmo texto é desenhado de **três maneiras**: no site com Recharts, no PDF com reportlab e no e-mail com barras feitas de células de tabela HTML (e-mail não executa JavaScript).

3.8 E-mails: modelos, variáveis e disparos

O Spillway é dono dos seus modelos de e-mail. O Relay é genérico e só preenche os valores.

Modelo	Variáveis por pessoa	Usado quando
Código de verificação	nome, código	Cadastro e reenvio
Redefinir senha	nome, código	Esqueci a senha
Confirmação de newsletter	link de confirmação, link de cancelar	Inscrição pelo formulário
Boas-vindas à newsletter	nome, link de cancelar	Ao ativar a newsletter
Relatório mensal	nome, link de cancelar	Todo mês e no momento da ativação
Análises	nome	Envio opcional por administrador

O **disparo mensal** (`POST /v1/admin/dispatches` ou o comando `python -m app.dispatch`) funciona assim:

- envia **uma vez por tipo e mês** (há uma tabela `email_dispatch` com restrição de unicidade);
- cada destinatário tem uma **chave de idempotência fixa** (tipo, mês, hash do e-mail): se o envio parar no meio, repete-se sem duplicar;
- o relatório vai **no corpo do e-mail**, abrindo com os quatro indicadores do mês (energia cortada, % de corte, perda estimada, usinas afetadas) e a variação contra o mês anterior.

3.9 Previsão publicada pelo banco

O site mostra a previsão lendo `forecast_run` e `forecast_hourly` no próprio Postgres. **A API nunca chama o serviço de ML**. Isso é explicado em detalhe no capítulo 8 e na decisão 7 (capítulo 11).

3.10 Migrações do banco

Versão	O que criou
0001	Esquema inicial (usinas, curtailment mensal, relatórios, newsletter)
0002	Membros e sessões
0003	Perfil horário
0004	Eventos de uso
0005	Contagem de tentativas de código
0006	`email_dispatch` (envios mensais)
0007	Tabelas da previsão (`forecast_model` , `forecast_run` , `forecast_hourly`)

> **Atenção:** em produção, uma migração nova só passa a existir depois do deploy da API. A permissão de escrita nas tabelas novas também precisa estar no `spillway-infra` (os scripts de permissão classificam toda tabela).

3.11 Endpoints do sistema elétrico, da água e da rede

Todos públicos, com cache e limpos pela revalidação. As tabelas e colunas estão no contrato (`docs/phase1-contract.md`).

Rota	Para que serve
`GET /v1/system/balance`	balanço horário por subsistema (hidráulica, térmica, eólica, solar, carga, intercâmbio), até 31 dias
`GET /v1/system/generation`	geração mensal por fonte, com energia, média, pico e participação
`GET /v1/plants/{slug}/generation`	geração mensal de uma usina e fator de capacidade
`GET /v1/system/interchange` e `/hourly`	intercâmbio entre subsistemas e com outros países
`GET /v1/system/thermal-dispatch`	geração térmica por motivo do despacho
`GET /v1/hydro/subsystems`	energia armazenada (EAR) e afluente (ENA) por subsistema, por dia
`GET /v1/hydro/reservoirs` e `/{slug}`	reservatórios e a série diária de um deles
`GET /v1/hydro/spilled-energy`	energia vertida que poderia ter sido turbinada
`GET /v1/grid/lines` e `/substations`	linhas e subestações, com filtros de tensão e estado e `as_of` para ver o cadastro numa data

> **Por quê `power_plant` e não a tabela `plant`?** A `plant` só aceita eólica e solar (é a base do produto de curtailment). A frota inteira (hidrelétricas, térmicas, nuclear) ganhou tabela própria, com o mesmo id e a mesma regra de slug.

As migrações 0008 a 0012 criam essas tabelas; a 0012 remove a checagem de faixa da energia armazenada por reservatório, que pode ser negativa.

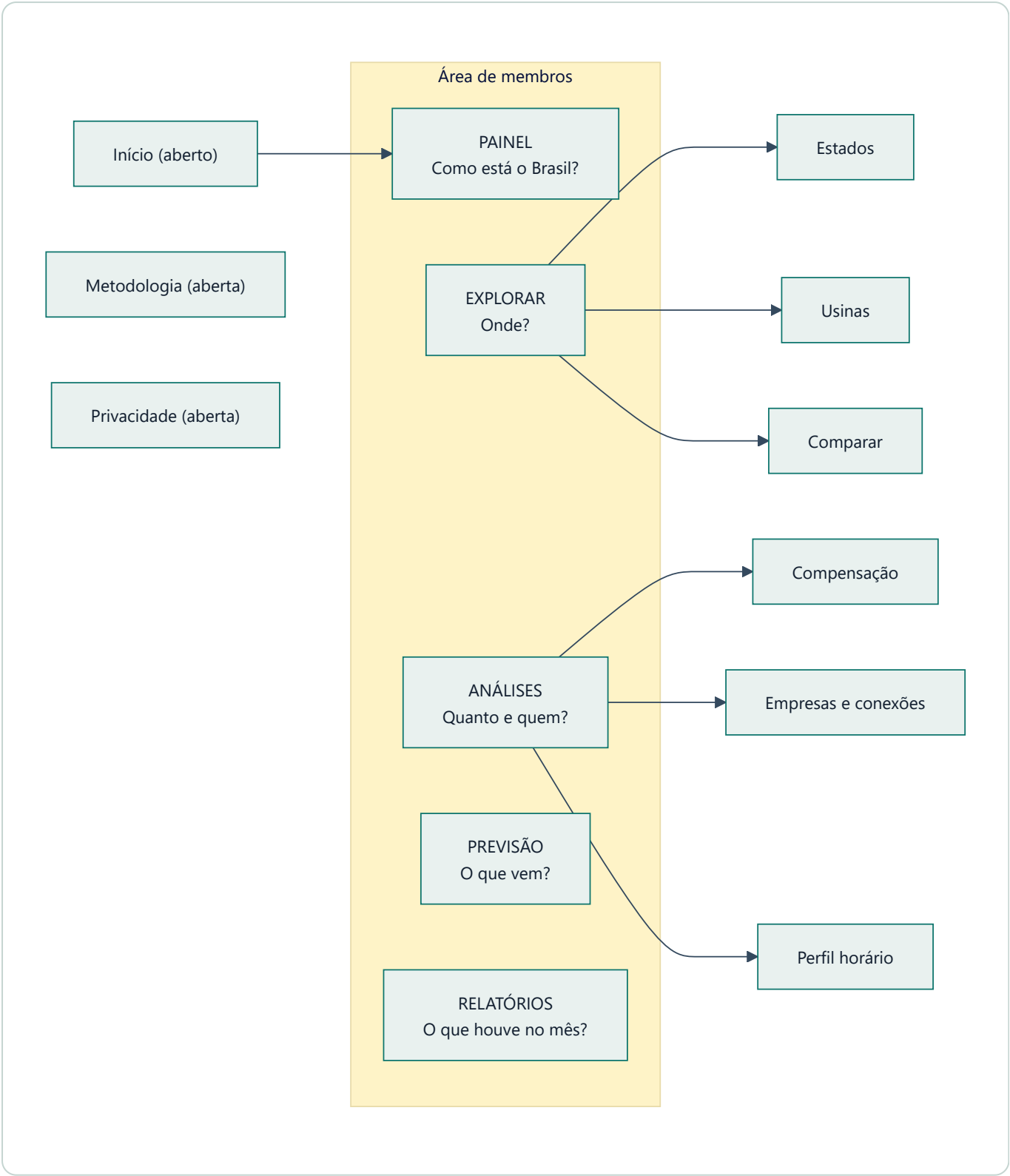
4. spillway-web: o site

4.1 Papel e tecnologia

É a cara do produto: **Next.js 15** (App Router, React), **Tailwind v4** para o visual, **Recharts** para os gráficos, tema claro e escuro. Fala **somente com a** `spillway-api`; nunca com o banco. Hospedado na **Vercel** (fora da AWS).

4.2 O mapa do site (menu por pergunta)

O menu foi organizado pela pergunta que a pessoa traz, não pelo tipo de página:

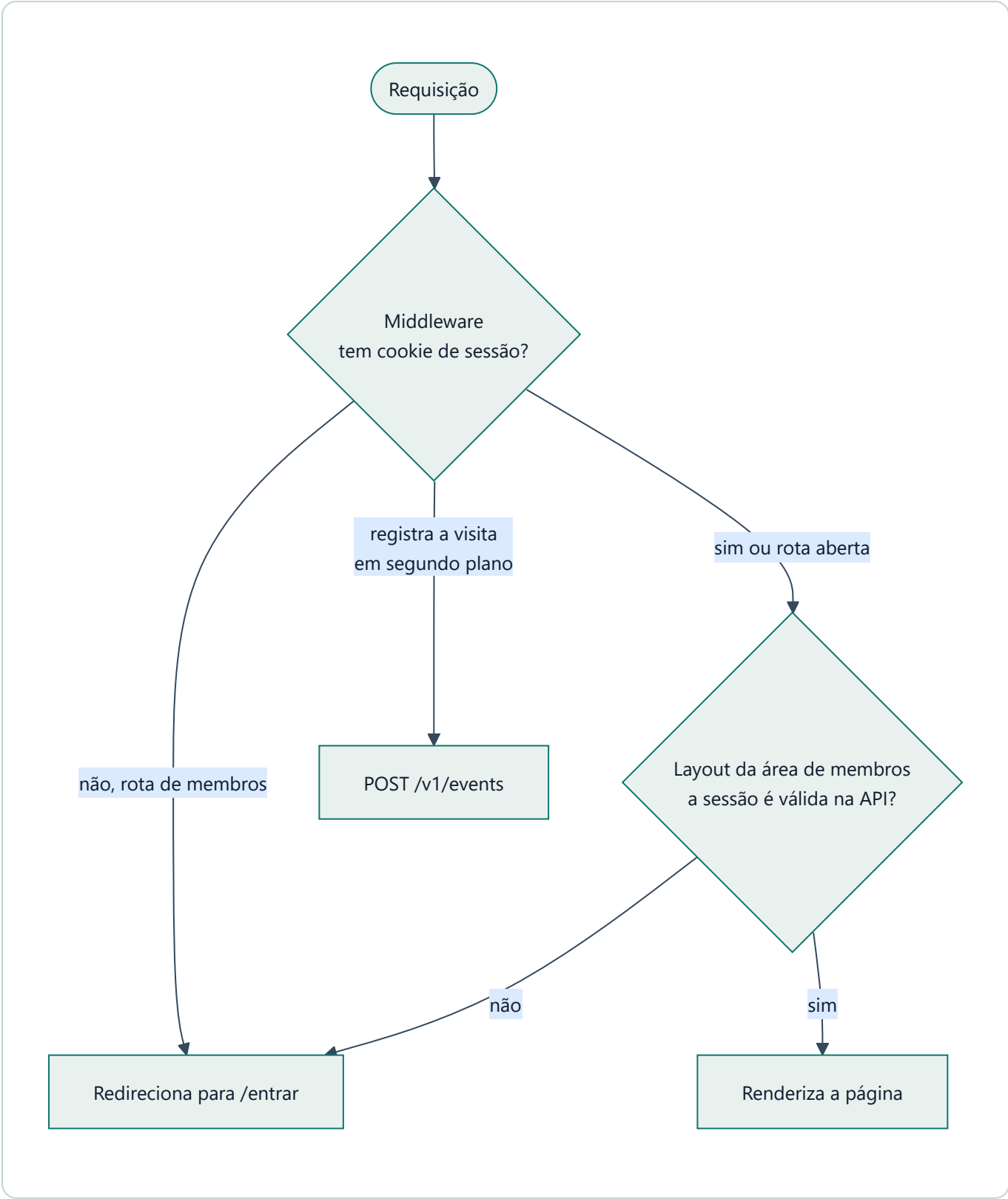


Seção	Pergunta	Conteúdo
Painel	"Como está o Brasil?"	Indicadores, séries por fonte e subsistema, filtro de período, link para o relatório do mês
Explorar	"Onde?"	Abas Estados, Usinas e Comparar (Comparar também é um botão nas páginas de estado e usina)

Seção	Pergunta	Conteúdo
Análises	"Quanto, quem e quando?"	Compensação, empresas e pontos de conexão, perfil horário, com **Baixar** (PDF ou dados)
Previsão	"O que vem?"	Mapa de calor do risco por hora e os resultados do treino do modelo
Relatórios	"O que houve no mês?"	Um relatório por mês, de 2021 em diante, com gráficos e PDF
Metodologia	"Como se calcula?"	Aberta a todos (rodapé)

> **Por quê?** A navegação antiga misturava geografia (Estados), entidade (Usinas), narrativa (Relatórios), lente analítica (Análises) e ação (Comparar). Agrupar por pergunta reduziu o menu de sete para cinco itens e tirou a redundância (o relatório mensal repetia o Painel em texto; agora o Painel aponta para ele).

4.3 Quem pode ver o quê



A verificação em duas etapas existe porque o cookie sozinho pode ser forjado. O **middleware** é rápido e barra a maioria. O **layout** confirma a sessão de verdade perguntando à API (`/v1/auth/me`).

4.4 Como as páginas carregam (e por que ficam rápidas)

Medições do agente de performance mostraram que **quase toda a lentidão local era o modo de desenvolvimento** (`next dev` compila cada página na primeira visita). Em produção, uma página responde em 35 a 90 ms. Mesmo assim, três técnicas ajudam:

Técnica	Efeito
`loading.tsx` na área de membros	Mostra um esqueleto na hora; o navegador consegue pré-buscar
Cache do roteador (30 s dinâmicas, 180 s estáticas)	Voltar a uma página visitada é instantâneo
Cache de `fetch` com tags	Os dados da API ficam por 1 hora e são invalidados pelo pipeline

> **Atenção:** no ambiente local, o cache de `fetch` do Next fica em `.next/cache/fetch-cache` e **persiste entre reinícios**. Depois de um backfill ou de apagar um relatório, apague essa pasta, senão o site mostra dados antigos. Em produção, o pipeline invalida as tags sozinho.

4.5 Rotas internas (`/api/*`)

Rota	Função
`/api/revalidate`	Recebe o aviso da API e invalida tags do cache
`/api/baixar/analise`, `/api/baixar/relatorio/[slug]`	Proxy de download: confere a sessão, pede o arquivo à API e entrega com o nome certo
`/api/session`	Dá ao cabeçalho o estado de login
`/api/track`	Registra eventos de uso
`/api/search`, `/api/auth/*`	Busca de usinas e autenticação do administrador

> **Por quê o proxy de download?** O navegador baixa do próprio site (mesma origem), a sessão é conferida no servidor e o limite de requisições da API enxerga o IP real do visitante.

4.6 Pontos de usabilidade que valem registrar

- **Newsletter na home:** quem está logado vê um botão para ativar o e-mail mensal (um clique, sem confirmação, porque o endereço já foi verificado). Depois de ativar, a home só diz "Você habilitou a newsletter mensal". O cancelamento fica na página "Cancelar newsletter" do rodapé.

- **Previsão:** abre com uma barra de progresso de 10 segundos, uma vez por sessão do navegador, com botão "Mostrar agora" e respeito à preferência de menos animação.
- **Tema:** o tema escuro vale mesmo antes do script de tema rodar, evitando o "flash" cinza ao recarregar.
- **Conta já existente:** o cadastro avisa claramente "já existe uma conta", com links para entrar e recuperar a senha.

4.7 Variáveis de ambiente principais

Variável	Para quê
`API_URL`	Onde está a API (local: `http://127.0.0.1:8000`)
`WEB_API_SECRET`	Segredo compartilhado para repassar o IP real do visitante
`REVALIDATE_SECRET`	Segredo da rota `/api/revalidate`
`NEXT_PUBLIC_SITE_URL`	URL pública (canônica, Open Graph)

4.8 Testes e qualidade

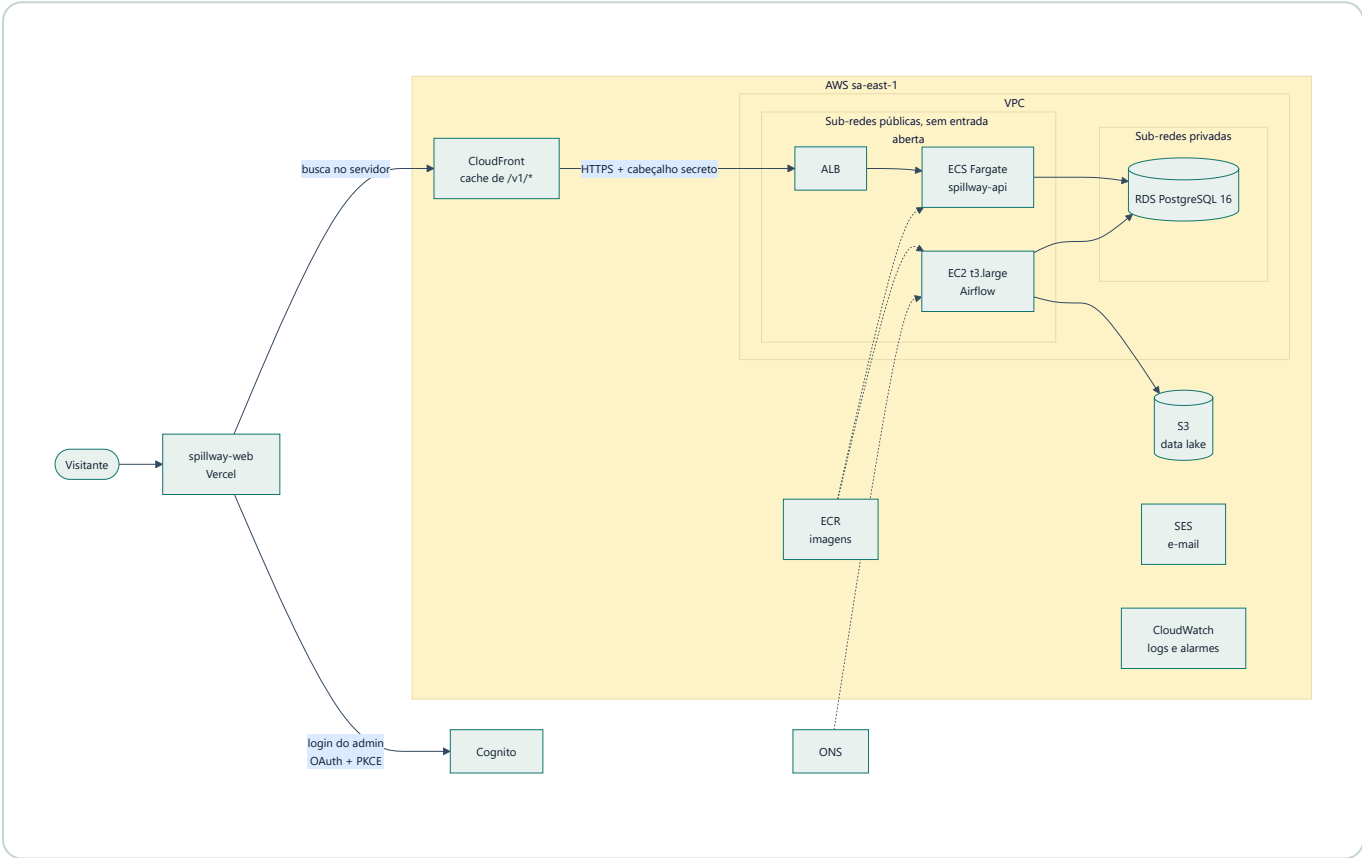
Vitest e Testing Library (cerca de 160 testes), ESLint, Prettier e TypeScript estrito. O CI também roda auditoria de dependências e varredura de segredos.

5. spillway-infra: a infraestrutura na AWS

5.1 Papel

Define **toda a infraestrutura do Spillway em código** (Terraform), na região **sa-east-1 (São Paulo)**. Nada é criado clicando no console: o que está no repositório é o que existe. O site em si roda na **Vercel**, fora da AWS.

5.2 A arquitetura



5.3 Módulos Terraform

Módulo	O que cria
`network`	VPC, 2 sub-redes públicas e 2 privadas em 2 zonas, gateway da internet, endpoint do S3
`lake`	Bucket do data lake: versionado, criptografado, sem acesso público; `bronze/` migra para armazenamento barato após 30 dias
`ecr`	Repositórios de imagens (API e pipeline), com varredura ao enviar

Módulo	O que cria
`database`	RDS PostgreSQL 16 criptografado, privado, backups de 7 dias, senha mestre gerenciada pelo RDS, usuários separados para a API e para o pipeline
`api`	ALB, cluster ECS, serviço Fargate (0,25 vCPU, 0,5 GB), tarefa de migração
`cdn`	CloudFront, políticas de cache
`auth`	Cognito: pool de usuários (MFA obrigatório), grupo `admin`, clientes para o site e para o pipeline
`airflow`	EC2 sem SSH e sem porta de entrada (acesso por SSM), com Docker
`email`	Identidade de domínio no SES (quando há domínio)
`observability`	Grupos de log, tópico SNS por e-mail, alarmes (5xx do ALB, tarefas, banco, falhas do Airflow)
`github_oidc`	Papéis para o GitHub Actions entrar na AWS **sem chaves fixas**

5.4 Os usuários do banco e as permissões

O banco tem dois usuários de aplicação com permissões **mínimas e separadas**:

Usuário	Pode
`energia_api`	Ler tudo; escrever somente nas tabelas da API (relatórios, membros, sessões, newsletter, envios mensais, eventos)
`energia_data`	Escrever somente nas tabelas que o pipeline carrega; **não consegue nem ler** dados de membros
`energia_forecast` (previsto)	Ler tudo; inserir e apagar nas três tabelas de previsão; atualizar apenas duas colunas de controle

Um **teste no CI** garante isso: ele sobe um Postgres, aplica as migrações da API e confere que cada usuário escreve só onde deve. Toda tabela nova precisa ser classificada, senão o CI falha. Foi assim que descobrimos que a tabela `email_dispatch` precisava de permissão para a API.

> **Por quê?** Se o pipeline for comprometido, ele não alcança os dados das pessoas cadastradas. Se a API tiver um bug, ela não consegue apagar os dados do ONS.

5.5 Decisões de infraestrutura

Por quê sem NAT gateway? Um NAT gateway custa em torno de US$ 40 por mês parado. Em vez dele, as tarefas ficam em sub-redes públicas **sem nenhuma porta de entrada aberta** (só aceitam tráfego vindo do ALB), o banco fica em sub-redes privadas e o S3 é acessado por um endpoint. A conta total fica na faixa de **US$ 170 a 270 por mês**, e a maior parte é a máquina do Airflow.

Por quê deploy sem chaves (OIDC)? Chaves de AWS guardadas no GitHub são um risco: vazam, não expiram. Com OIDC o GitHub Actions prova quem é e recebe uma credencial temporária. Cada repositório tem seu papel, e a aplicação do Terraform só pode ser assumida pelo ambiente `production`, que exige aprovação de uma pessoa.

Atenção: os nomes **implantados** continuam com o prefixo antigo `energia` (repositórios ECR, o escopo `energia-api/admin` do Cognito, os usuários `energia_api` e `energia_data`, o caminho `/opt/energia-data` na EC2). Renomear esses recursos exige migração de infraestrutura e foi deixado de fora de propósito. Além disso, depois da renomeação dos repositórios no GitHub, a confiança do OIDC aponta para os nomes novos, mas **só vale depois de um** `terraform apply` em produção.

5.6 O que falta para entrar no ar

O repositório tem um **roteiro de primeira implantação** (state bucket, variáveis, plan e apply, migração da API, usuários do banco, primeiro administrador no Cognito, Airflow na EC2, GitHub, Vercel e SES). Nada disso foi aplicado em produção ainda: **o Spillway roda hoje só no seu computador**.

5.7 Custo mensal estimado

Componente	Ordem de grandeza (US$/mês)
EC2 do Airflow + disco	90 a 120
RDS	20 a 35
ALB	20 a 30
Fargate (1 tarefa)	10 a 15
IPv4 público	15 a 25
CloudWatch, Secrets, S3, CloudFront, outros	15 a 40
Total	**170 a 270**

Atenção: é uma estimativa grosseira, não um orçamento. Confira na calculadora da AWS antes de decidir. Reduzir a máquina do Airflow depois de medir o uso é a economia mais óbvia.

6. Relay: a plataforma de mensagens

6.1 O problema que ele resolve

Enviar **uma** mensagem é fácil. Enviar milhões por dia exige responder a perguntas difíceis:

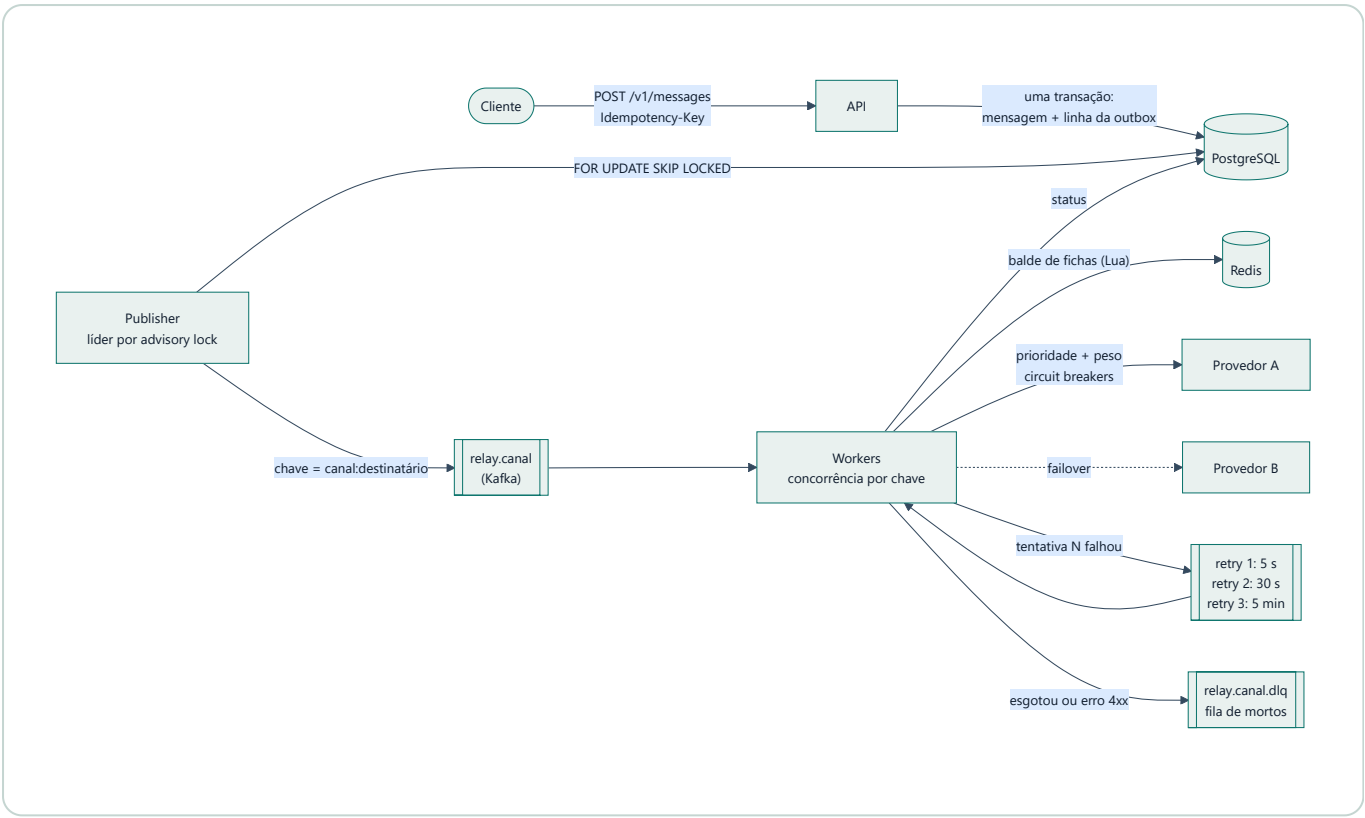
Pergunta	Resposta do Relay
O que acontece com uma mensagem aceita quando o Kafka está fora do ar?	Ela espera numa tabela do Postgres (a *outbox*) e **nunca se perde**.
E se o provedor de SMS cair no pico?	O tráfego **troca de provedor na mesma tentativa**.
E se o cliente repetir uma requisição que deu timeout?	A **chave de idempotência** devolve a mensagem original, sem enviar duas vezes.
Como 12 servidores respeitam um contrato de 1.500 msg/s?	**Um balde de fichas único no Redis**, compartilhado por todos.
Como dois códigos (OTP) para a mesma pessoa saem na ordem certa?	A chave do Kafka é **o destinatário**, então ficam na mesma partição.

Cada resposta é uma decisão com custos e benefícios, registrada em **ADRs** (oito no total).

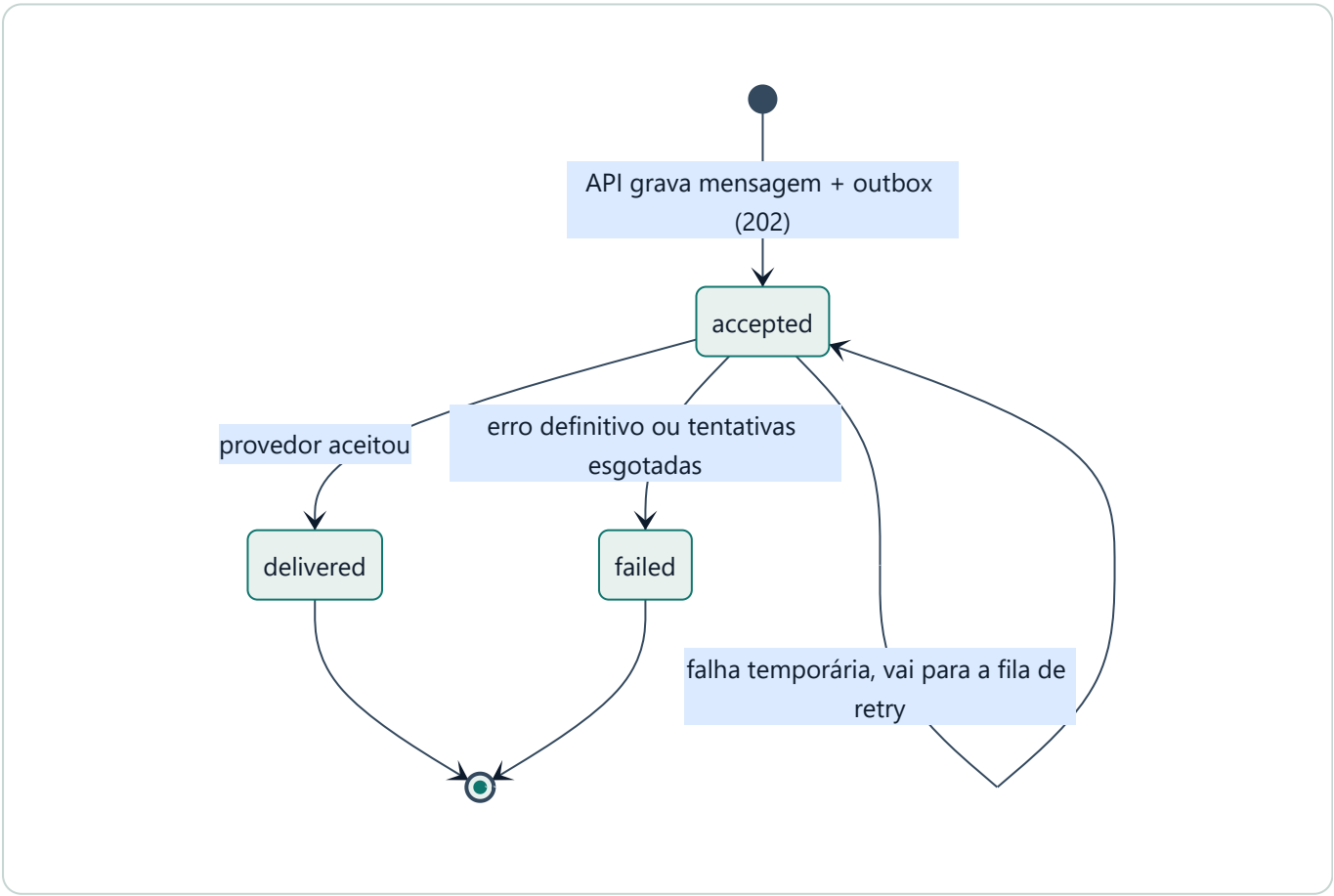
6.2 A arquitetura

Uma única imagem de software, quatro papéis escolhidos pela variável `APP_ROLE`:

Papel	O que faz	Como escala
`api`	Valida, grava mensagem e linha da outbox **numa transação** e responde `202`	Horizontal (sem estado)
`publisher`	Esvazia a outbox para o Kafka em lotes	2 réplicas, 1 ativa (trava de líder)
`worker`	Consome, limita a taxa, escolhe o provedor, tenta de novo, manda para a fila de mortos	Horizontal, até o número de partições
`migrate`	Aplica as migrações do banco com uma trava	Tarefa pontual

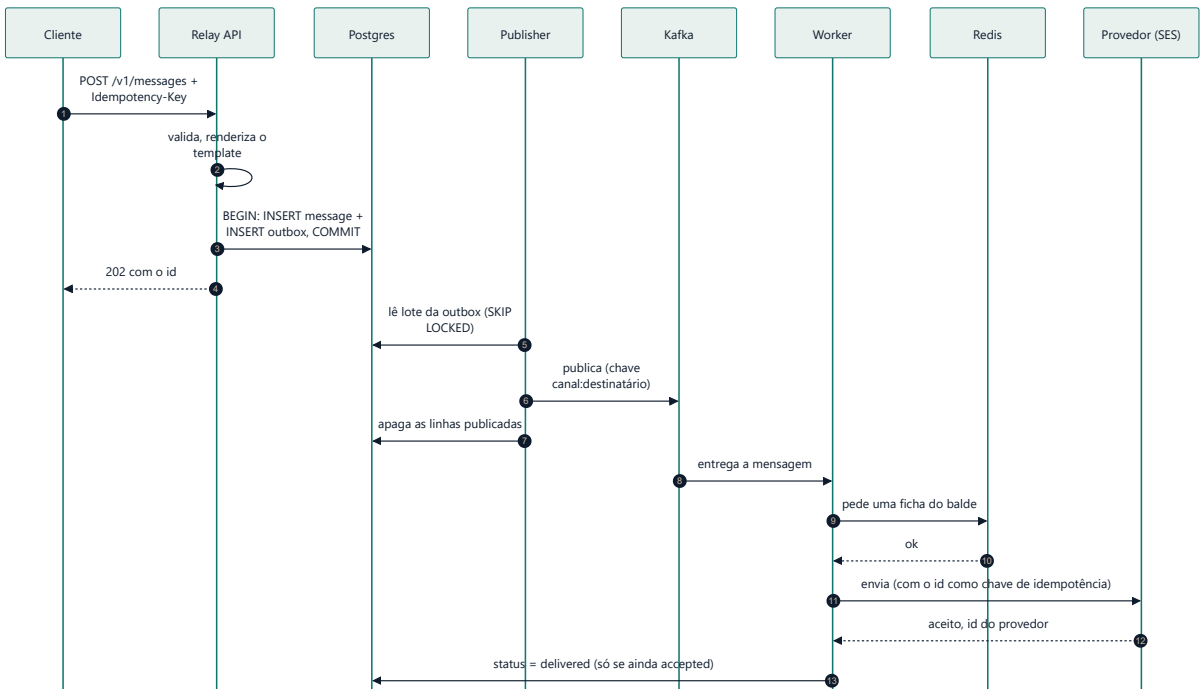


6.3 A vida de uma mensagem



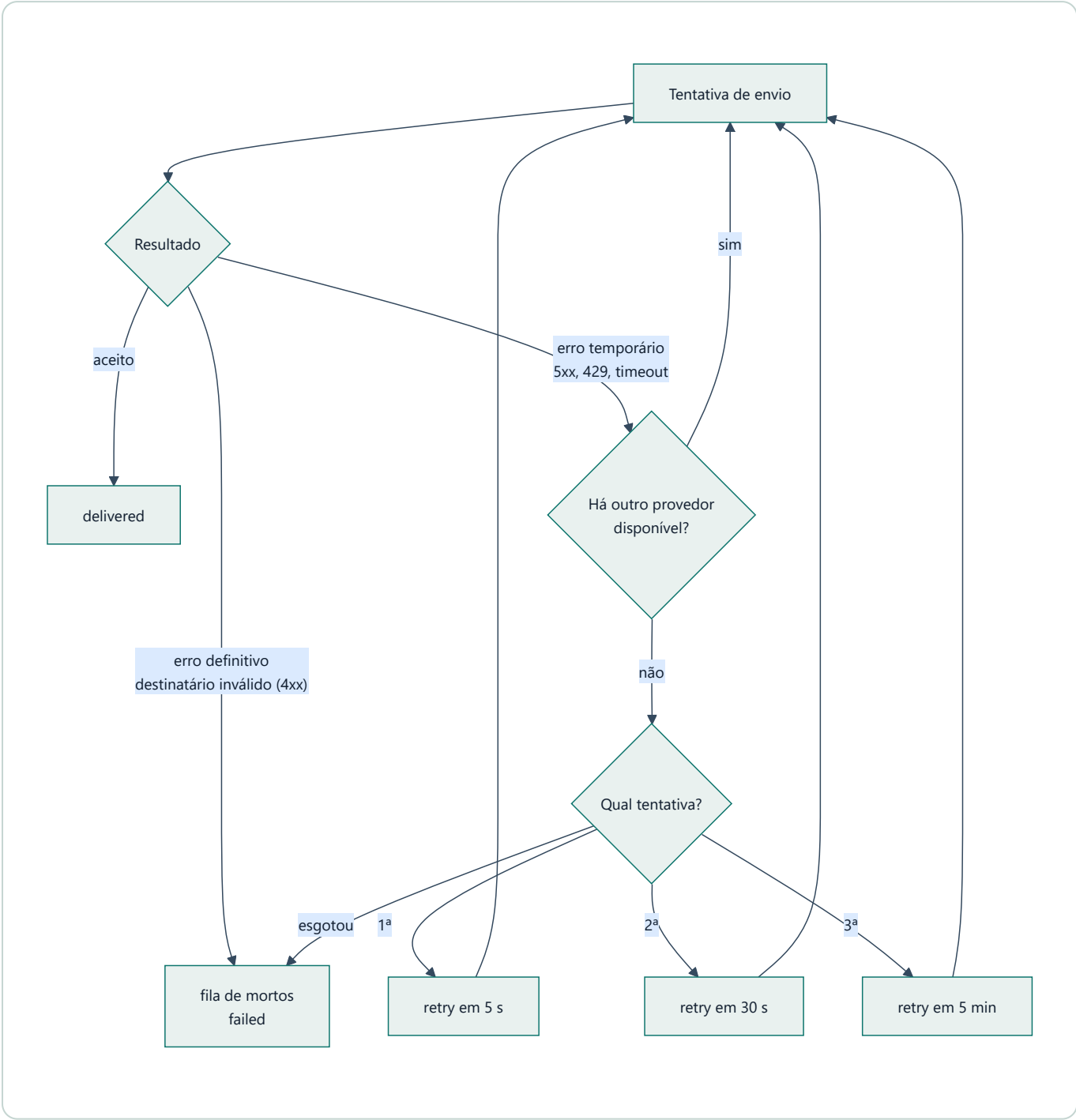
`accepted`, `delivered` e `failed` dizem **o que o Relay fez**. O que o destinatário recebeu de fato é outra dimensão, o `delivery_status` (capítulo 7).

6.4 Sequência de um envio feliz



> **Por quê a outbox?** Se a API escrevesse no banco **e** no Kafka separadamente, uma queda entre os dois perderia ou duplicaria mensagens. Escrevendo só no Postgres (uma transação) e deixando um processo separado levar para o Kafka, a disponibilidade da API depende apenas do banco. A mensagem aceita é uma promessa cumprida.

6.5 Falhas e novas tentativas



- **Filas de retry em camadas:** três tópicos, cada um com **um atraso fixo**. Como todas as mensagens de uma camada esperam o mesmo tempo, elas vencem na ordem de chegada. O consumidor olha a primeira, e se ainda não venceu, **pausa a partição** até vencer. Sem temporizador por mensagem.
- **Mensagem malformada** (que nem dá para ler) vai direto para a fila de mortos e nunca trava uma partição.
- **Fila de mortos** guarda o erro para análise e a mensagem fica `failed`.

> **Por quê?** Um cronômetro por mensagem (ou um `sleep` no consumidor) não escala. Camadas com atraso fixo transformam "espere 30 s" em "pause a partição até o primeiro item vencer".

6.6 Troca de provedor e circuit breaker

Cada canal tem uma lista de provedores com **prioridade** e, dentro da mesma prioridade, **peso** (por exemplo, 75% e 25% entre dois gateways de SMS). Cada tentativa percorre a lista: se o primeiro falha de forma recuperável, o próximo é usado **na mesma tentativa**.

Cada provedor tem um **circuit breaker** (disjuntor):

Parâmetro	Valor
Janela	50 chamadas
Mínimo para decidir	20 chamadas
Abre com	50% de falhas
Meio aberto depois de	15 s, com 3 chamadas de prova

Erros do cliente (destinatário inválido) e a limitação local **não** contam contra a saúde do provedor.

> **Por quê?** Sem disjuntor, um provedor morto custa um timeout por mensagem. Com ele, custa uma verificação rápida. Os disjuntores vivem **em cada processo**, sem estado compartilhado: simples e sem ponto único de falha, ao preço de cada servidor descobrir a falha por conta própria.

6.7 Limite de taxa para toda a frota

Os provedores têm contratos por conta (por exemplo, 1.500 SMS por segundo), não por servidor. Com servidores que sobem e descem, um limite por processo estaria errado por um fator igual ao número de réplicas.

- Um **balde de fichas por provedor no Redis**, compartilhado.
- O "pega uma ficha" é um **script Lua**, executado de forma atômica pelo Redis: dois workers nunca gastam a mesma ficha.
- O worker espera até **250 ms** por uma ficha. Passando disso, a tentativa falha de forma recuperável e o roteador **transborda para o próximo provedor**. Um primário saturado manda o excesso para o secundário, em vez de fazer fila.
- Se o Redis cair, os workers **param de entregar**, em vez de violar o contrato: é a escolha conservadora para um recurso pago e limitado.

6.8 Idempotência em cada salto

O sistema é **pelo-menos-uma-vez**, e cada salto remove as duplicatas que consegue:

Salto	Origem da duplicata	Proteção
Cliente para API	Cliente repete	Cabeçalho `Idempotency-Key`, único no Postgres. Mesma chave e mesmo conteúdo: devolve o original (`200`). Mesma chave e conteúdo diferente: `409`.
Outbox para Kafka	Lote republicado	Produtor idempotente
Kafka para worker	Reentrega após queda	O worker ignora o que já está `delivered` ou `failed`, e só atualiza `WHERE status = 'accepted'`
Worker para provedor	Queda depois de o provedor aceitar	O id da mensagem vai como chave de idempotência

> **Atenção:** a única janela restante é inerente a provedores externos: um worker que morre depois de um provedor **sem suporte a chave de idempotência** (SES e Twilio não têm) aceitar a mensagem. O risco é um duplicado raro, documentado.

6.9 Ordem por destinatário

As mensagens são chaveadas por `canal:destinatário`, então uma pessoa cai sempre na mesma partição. Dentro de um lote, um **executor por chave** processa destinatários diferentes em paralelo (até 64) e o mesmo destinatário em sequência. Os deslocamentos (offsets) só são confirmados depois do lote inteiro, então uma queda **reprocessa** em vez de pular.

Uma mensagem que passou por retry pode ser ultrapassada por uma mais nova para a mesma pessoa, que é exatamente o que se quer para códigos de uso único (OTP).

6.9.1 Templates e campos dinâmicos

O Relay **não guarda modelos**. Cada produto manda o seu HTML e os valores (ADR 8):

```
{
  "channel": "email",
  "to": "ana@example.com",
  "subject": "Seu código: {{code}}",
  "body": "Olá {{name}}, seu código é {{code}}.",
  "html": "<h1>Olá {{name}}</h1><p>Código <b>{{code}}</b></p>",
  "variables": { "name": "Ana", "code": "482913" },
  "metadata": { "product": "energia", "template": "verification-code" }
}
```

- `{{nome}}` é substituído em **uma passada** (um valor que contém `{{outro}}` não é expandido de novo).
- Dentro do HTML, os valores são **escapados**: um nome como `<script>` nunca vira código na caixa de entrada.
- Placeholder sem variável é erro `400`, não um e-mail com `{{code}}` escrito.
- O `html` só vale para e-mail. O texto continua obrigatório (é a alternativa em texto).

> **Por quê?** Cada produto tem seus e-mails e seu visual. O Relay não deve conhecer nenhum deles, nem virar um catálogo que cada produto precise "implantar" antes de enviar. Mudar um e-mail não exige deploy do Relay.

6.10 Provedores reais

Provedor	Canal	Observações
Amazon SES	E-mail	Usa a cadeia padrão de credenciais da AWS. As tentativas do SDK são desligadas (o Relay faz as suas). O id da mensagem vai numa *tag* do SES para ligar bounces e reclamações de volta.
Twilio	SMS e WhatsApp	Erros mapeados em definitivos (número inválido, descadastrado) e recuperáveis (limite, 5xx)
Genérico HTTP	Qualquer	`POST {url}/v1/send`. Usado pelos provedores simulados do ambiente local, e fácil de colocar na frente de um gateway interno

> **Atenção:** a conta do SES está no **modo sandbox**: só envia para endereços verificados (hoje, o seu). Para enviar a outras pessoas é preciso pedir a saída do sandbox na AWS.

6.11 A API do Relay

Método	Rota	Para quê
`POST`	`/v1/messages`	Enviar. `202` com `{id, status}`. `200` com `Idempotent-Replayed: true` para repetição igual. `409` para mesma chave com conteúdo diferente. `400` lista todos os erros de validação. `422` se o destinatário está na lista de supressão.
`GET`	`/v1/messages/:id`	Status, tentativas, provedor usado, último erro, status de entrega e recibos
`POST`	`/v1/receipts`	Recibo de entrega neutro (capítulo 7)
`DELETE`	`/v1/suppressions/:canal/:destinatário`	Tira alguém da lista de supressão

Método	Rota	Para quê
`GET`	`/health/live`, `/health/ready`	Saúde (processo e dependências)
`GET`	`/metrics`	Prometheus

Com a variável `API_KEYS` definida, toda chamada exige `X-Api-Key`.

6.12 Observabilidade

Prometheus coleta as métricas de cada papel e o Grafana (porta 3030) tem painéis prontos: mensagens aceitas, entregues, tentativas, estado dos disjuntores, idade da outbox, recibos recebidos. O script de carga lê as próprias métricas do serviço, então o relatório mede **a entrega ao provedor**, não só "a API disse aceito".

6.13 Números medidos

Teste de carga com k6, tudo rodando no mesmo notebook (Intel i7-7700HQ, 4 núcleos, 16 GB), com provedores simulados, **1 API, 1 publisher e 2 workers**:

Métrica	250 msg/s por 2 min	500 msg/s por 2 min
Aceitas (202)	29.920 de 30.000	35.585 de 60.000
`POST /v1/messages` p50 / p95	18 ms / 210 ms	1,48 s / 2,65 s
Entregues ao provedor	29.920 (100% das aceitas)	todas, depois do teste
Entrega ponta a ponta p50 / p95 / p99	258 ms / 709 ms / 1,43 s	não medido
Idade máxima da outbox	394 ms	não medido

> **Honestidade:** uma API sozinha satura perto de **290 mensagens aceitas por segundo** neste notebook (cada requisição é uma transação no Postgres). A API não tem estado, então escala horizontalmente. Numa corrida anterior, a entrega ficou parada porque meu `.env` local só consumia o canal de e-mail; ao liberar todos os canais, **dois workers esvaziaram 49 mil mensagens em cerca de 90 s (≈ 540 msg/s)**. Os limiares do k6 foram ultrapassados nas duas corridas, e os números foram publicados como saíram.

6.14 Como subir localmente

```
cd relay
docker compose up -d --build api publisher worker postgres redis redpanda
curl localhost:3000/health/ready
```

Serviço	Porta
API	3000
Publisher (métricas)	3001
Redpanda (Kafka)	19092
Postgres	5432
Prometheus / Grafana	9090 / 3030
Provedores simulados (SMS, e-mail, WhatsApp, push)	8091 a 8096

> **Dica:** `PROVIDERS=providers.real.json` troca os simulados pelo SES real; `WORKER_CHANNELS` controla quais canais os workers consomem. Para testes de carga, use sempre os simulados.

Há também manifestos **Kubernetes** (`k8s/`) com sondas, requisições de recursos, contêineres sem root, PodDisruptionBudgets e HPAs.

7. Os satélites do Relay: gateway, recibos e Terraform

7.1 relay-api-gateway: a porta de entrada simples

É um servidor Node **sem dependências** que fica na frente do Relay. Quem quer enviar mensagens (como a `spillway-api`) fala com ele, e ele repassa para a API do Relay.

Responsabilidade	Detalhe
Autenticação	`X-Api-Key` dele (`GATEWAY_API_KEYS`); usa outra chave para falar com o Relay
Validação rápida	Formato da requisição e canais habilitados (`ALLOWED_CHANNELS`); o resto da validação continua sendo do Relay, que é a fonte da verdade
Passagem	Repassa `Idempotency-Key`, `html`, `variables` e `metadata`; devolve os erros do Relay sem alterar
Limite de corpo	512 KB (cabe um HTML de e-mail grande)

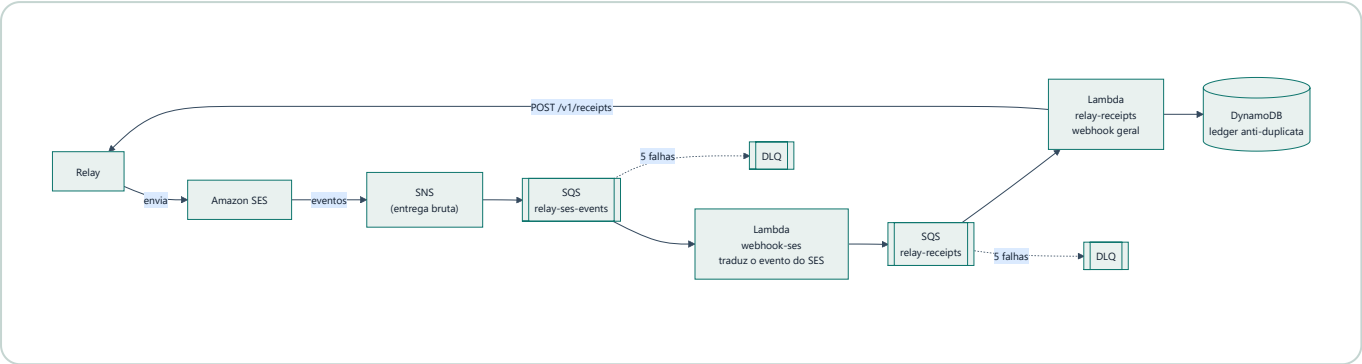
> **Por quê um gateway, e não chamar o Relay direto?** O gateway **não pula o Relay**: ele repassa para a mesma API, então a outbox e a idempotência continuam valendo. A vantagem é separar "quem pode enviar" (chaves por produto) de "como se envia" (o Relay), e poder expor só o gateway.

7.2 relay-receipts: o que o provedor diz depois

Aceitar uma mensagem não é entregá-la. O SES pode devolver o e-mail minutos depois (*bounce*), ou a pessoa pode marcar como spam (*complaint*). Isso precisa voltar para o Relay para duas coisas:

1. **Atualizar o status de entrega** da mensagem;
2. **Parar de enviar** para endereços que devolvem ou reclamam, porque isso destrói a reputação do remetente.

O caminho do recibo



Duas funções Lambda em **Python**, com papéis bem separados:

Função	Conhece	Faz
`webhook-ses`	O formato do **SES**	Lê o evento bruto, valida e transforma num **recibo neutro** (ver abaixo), e coloca na segunda fila
`relay-receipts`	**Nenhum provedor**	É o "webhook geral": pega qualquer recibo neutro, confere num ledger (DynamoDB) se já viu, e envia para o Relay

Para um novo provedor (Twilio, por exemplo), basta escrever um tradutor pequeno que publica na **mesma fila**. O `relay-receipts` e o Relay não mudam.

O recibo neutro

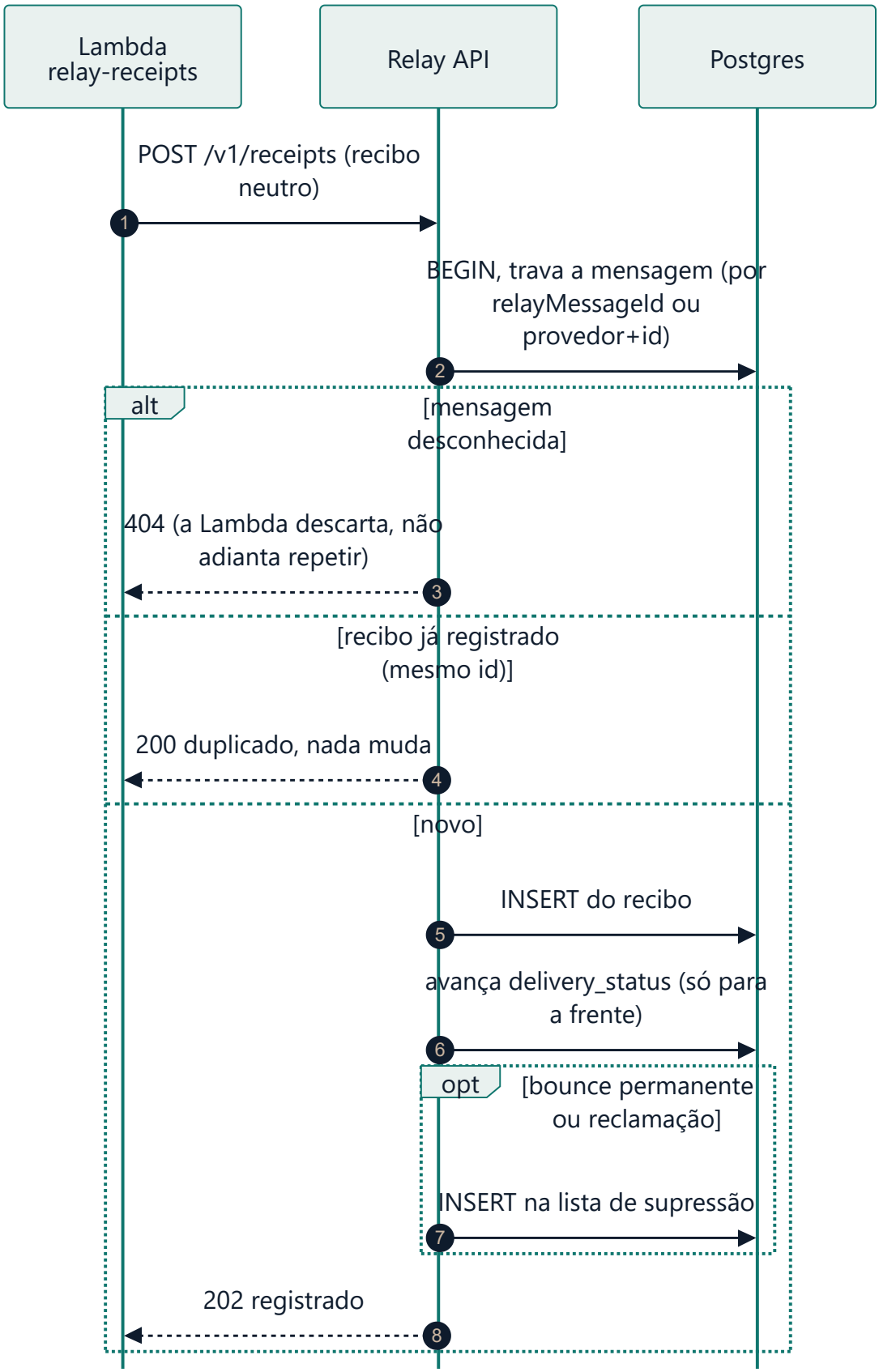
Um formato único, independente do provedor:

```
{
  "schema": 1,
  "id": "ses:7f3c...",
  "provider": "ses",
  "providerMessageId": "01030...",
  "relayMessageId": "01a182e5-...",
  "channel": "email",
  "recipient": "ana@example.org",
  "type": "bounced",
  "permanent": true,
  "reason": "smtp; 550 5.1.1 user unknown",
  "occurredAt": "2026-10-28T14:04:01Z"
}
```

Tipos: `sent`, `delivered`, `delayed`, `bounced`, `complained`, `rejected`. O `id` é derivado do evento do provedor, então o mesmo evento sempre gera o mesmo id.

7.3 O que o Relay faz com o recibo

`POST /v1/receipts` (ADR 7):


Lambda
relay-receipts
Relay API
Postgres
POST /v1/receipts (recibo neutro)
1
BEGIN, trava a mensagem (por relayMessageId ou provedor+id)
2
alt
[mensagem desconhecida]
404 (a Lambda descarta, não adianta repetir)
3
[recibo já registrado (mesmo id)]
200 duplicado, nada muda
4
[novo]
INSERT do recibo
5
avança delivery_status (só para a frente)
6
opt
[bounce permanente ou reclamação]
INSERT na lista de supressão
7
202 registrado
8

Status que só anda para a frente

Os eventos chegam **fora de ordem**. Por isso o `delivery_status` tem uma hierarquia e nunca regride:

`sent` < `delayed` < `delivered` < `bounced` ou `rejected` < `complained`

Um `sent` atrasado não desfaz um `delivered`.

Lista de supressão

Um *bounce* permanente ou uma reclamação põe o destinatário na tabela `suppressions`. Depois disso, `POST /v1/messages` para ele responde **422** antes de gravar qualquer coisa. Um bounce temporário **não** suprime. `DELETE /v1/suppressions/:canal/:destinatário` libera de novo. E-mails são comparados em minúsculas; telefones não são alterados.

> **Atenção:** só o envio **novo** é barrado. Mensagens que já estavam na fila antes do bounce ainda saem.

7.4 Por que SQS entre as duas Lambdas, e não API Gateway

Critério	SQS (escolhido)	API Gateway
Relay fora do ar	A fila **guarda** (14 dias)	O recibo falha ou se perde
Tentativas e fila de mortos	Prontas	A construir
Pico de eventos	O limite de concorrência protege o Relay	O Relay recebe tudo de uma vez
Exposição pública	Nenhuma	Um endpoint a autenticar e proteger
Custo	Menor nessa escala	Maior

O API Gateway continua sendo a ferramenta certa para provedores que **empurram** para nós (como o Twilio). O tradutor dele validaria a assinatura e publicaria na mesma fila.

7.5 Logs estruturados

As Lambdas escrevem **uma linha JSON por fato** no CloudWatch (retenção de 14 dias). Cada linha tem nível, mensagem, qual função e o id da invocação. Cada recibo ganha a sua linha com `receipt_id`, provedor, ids, canal, tipo, motivo e **destinatário mascarado** (`a***@example.org`), para os logs não virarem uma lista de endereços.

Mensagem	Quando
`receipt queued` / `receipt not queued`	O tradutor enfileirou (ou falhou ao enfileirar)
`ses event ignored`	Tipo de evento que o Relay não acompanha

Mensagem	Quando
`receipt delivered to relay`	O Relay respondeu `recorded` ou `duplicate`
`receipt not usable by relay`	`unmatched` ou `rejected`
`relay unavailable, will retry`	O Relay falhou
`receipt already handled`	O ledger já tinha visto

Para seguir uma mensagem pelas duas funções, use o CloudWatch Logs Insights filtrando por `relay_message_id`.

7.6 relay-terraform: tudo da AWS do Relay em código

Arquivo	Cria
`main.tf`	Identidade do SES, *configuration set*, destino de eventos, tópico SNS, fila `relay-ses-events` e a DLQ
`dns.tf`	Registros DKIM, SPF e DMARC (quando há zona no Route 53)
`iam.tf`	Política de envio, usuário e chave opcionais, papel opcional para EKS
`receipts.tf`	Fila `relay-receipts` e DLQ, tabela DynamoDB, as duas Lambdas, papéis, gatilhos, alarmes

Já aplicado na conta , região **sa-east-1**: 2 Lambdas, 4 filas, 1 tabela e 2 alarmes. O código das funções vem do repositório `relay-receipts`, empacotado pelo Terraform.

> **Atenção (estado atual):** a Lambda `relay-receipts` está **desligada de propósito** (o gatilho SQS está desabilitado) porque o Relay roda só no seu computador, e a AWS não o alcança. Os recibos ficam guardados na fila (14 dias). Para ligar, o Relay precisa de uma URL pública protegida por chave de API e depois `terraform apply -var relay_url=...`. A DLQ de eventos do SES tem 16 mensagens antigas (restos de testes) e por isso um alarme em estado ALARM.

8. spillway-forecast: a previsão de risco de corte

8.1 O que ele prevê

Para cada hora dos próximos dias, em cada **série** (estado ou subsistema × fonte eólica ou solar), o modelo estima duas coisas:

1. **O risco**: a probabilidade de aquela hora ter corte de pelo menos **10% da geração potencial**. É um indicador estatístico, não uma garantia.
2. **O corte esperado**: o percentual de corte esperado para a hora.

São **27 séries** modeladas (a solar do Mato Grosso do Sul tem menos de 120 dias de histórico e fica de fora).

> **Em resumo:** é um "termômetro de risco" por hora, apoiado em cinco anos de histórico, calendário e no comportamento recente. Ele **não usa clima**.

8.2 Os dados

Fonte	Período	Origem
Eólica	2021-10 até hoje	ONS (conjuntos), pela camada **silver** do `spillway-data`
Solar	2024-04 até hoje	idem (a ONS só publica a solar desde essa data)

As linhas de 30 minutos viram **séries horárias** por estado, subsistema e Brasil. A "geração potencial" é a geração verificada mais a cortada; o percentual de corte divide uma pela outra. Horas em que o denominador é menor que 1 MWh (a solar à noite) são **excluídas** do treino e da avaliação, e aparecem como "não aplicável".

O resultado do ajuste: cerca de **1,6 milhão de linhas de previsão** (uma origem a cada 3 dias × 168 horas à frente), das quais ~218 mil formam o teste final.

> **Atenção (a limitação mais importante hoje):** a camada silver só tem **meses completos** e o ONS publica o mês anterior no começo do seguinte. Por isso o último dado hoje é **30/09/2026**, e a previsão publicada cobre **1 a 7 de outubro**, ou seja, não é "a próxima semana a partir de hoje". A primeira versão do projeto, que baixava direto do ONS, chegava a 27/10 porque o ONS atualiza o **mês corrente** aos poucos. A correção, já mergeada, é a **ingestão híbrida**: silver para o histórico validado + cauda do mês corrente direto do ONS, marcada como provisória. Em 28/10/2026 os dados iam até 27/10 e a janela publicada ia de 28/10 a 03/11. Nos meses em comum, silver e ONS diferem só por ruído de ponto flutuante.

8.3 O que o modelo recebe (as variáveis)

Grupo	Exemplos
Calendário da hora prevista	hora, dia da semana, mês, fim de semana, **feriado brasileiro**, sazonalidade (seno e cosseno)
Histórico do corte (só o passado)	% de corte no último dia, 7, 28 e 90 dias; tendência; pico horário do último dia
Geração	nível de geração em 1, 7 e 28 dias
Mesmo horário em dias anteriores	ontem, média de 7 e de 28 dias, mesma hora na semana anterior
Identificação	a série e a fonte (variáveis categóricas)

Não usa clima (vento, irradiação), preço, consumo nem informação de indisponibilidade da rede.

8.4 O modelo e por que ele

São **dois** estimadores `HistGradientBoosting` do scikit-learn, treinados juntos nas mesmas variáveis: um **classificador** (o risco) e um **regressor** (o percentual esperado). Um modelo **global** para as 27 séries, com a série e a fonte como variáveis. Parâmetros: 250 iterações, taxa de aprendizado 0,06, 31 folhas, mínimo de 200 amostras por folha, regularização L2 igual a 1, sem parada antecipada, semente 42.

> **Por quê gradient boosting (árvores)?** É a família que costuma vencer em **dados tabulares com variáveis misturadas** (números e categorias) e interações como "de dia, em abril, no Nordeste, a eólica sofre mais". Aprende essas interações sozinho, é **rápido** com 1,6 milhão de linhas, lida com valores ausentes e produz **probabilidades bem calibradas** (o erro de calibração medido foi 0,036). Um único modelo global **aprende entre séries**: o padrão de uma região ajuda outra com menos histórico.

> **Por quê o HistGradientBoosting, e não LightGBM ou XGBoost?** Foi escolhido por instalar sem atrito no Windows e **não acrescentar dependências** além do scikit-learn. Para este tamanho de dado, o desempenho esperado é equivalente.

> **Por quê não modelos por série (Prophet, ARIMA)?** Seriam 27 modelos independentes, sem aprendizado compartilhado e com dificuldade de usar tantas variáveis de contexto.

> **Por quê não redes neurais (LSTM, Transformer)?** Com poucos anos de dados horários e sem variáveis externas ricas, elas raramente superam árvores, exigem mais engenharia e são mais difíceis de explicar e de calibrar.

> **Honestidade:** esse raciocínio é a justificativa de projeto. **Os únicos comparativos medidos são contra duas linhas de base simples** (abaixo). Não foram feitos testes de bancada contra Prophet, LightGBM ou redes neurais. Se isso importar para a pós ou para uma apresentação, vale executar esse comparativo.

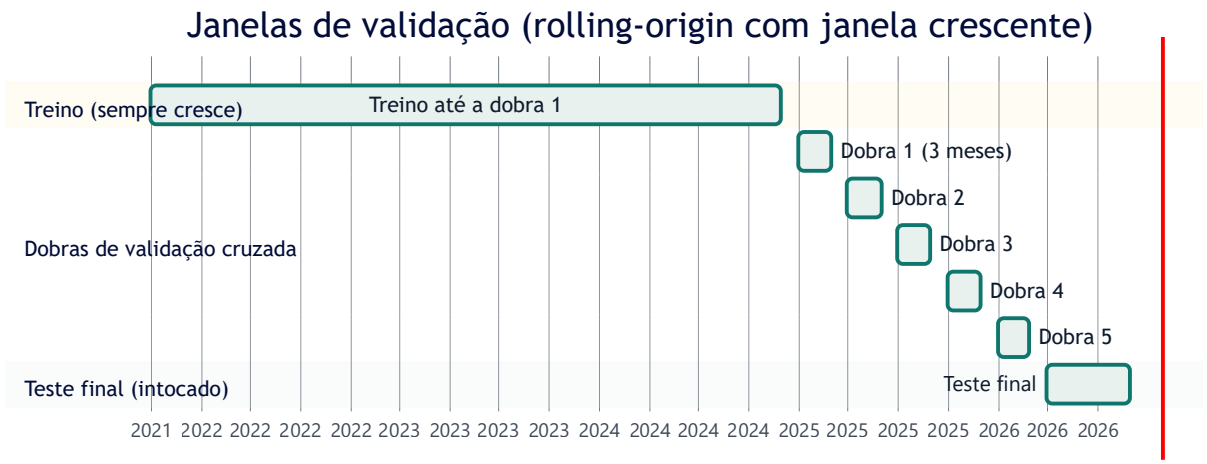
8.5 As linhas de base (contra o que o modelo é comparado)

Um modelo só vale se vencer alternativas simples. Foram usadas duas:

Linha de base	Ideia
Climatologia	O risco típico por série × mês × tipo de dia (útil ou fim de semana) × hora, aprendido do passado
Sazonal ingênuo	"A mesma hora na semana anterior"

8.6 Como se valida (sem se enganar)

Prever o futuro exige **validação temporal**: nada de sorteio aleatório, que deixaria o modelo "ver" o futuro.



Regras que garantem honestidade:

- **5 dobras de 3 meses** (2025-01 a 2026-03), cada uma treinando só com o que veio **antes** e com **7 dias de folga** (*embargo*) entre treino e teste.
- **Teste final intocado** (2026-04 a 2026-09): hiperparâmetros fixos, nada ajustado nele.

- **Teste de vazamento** automatizado: se alguém corromper todos os dados a partir da origem da previsão, nenhuma variável de histórico pode mudar.
- **Significância por blocos de semanas**: as linhas de treino se sobrepõem muito (cada origem gera 168 horas), então o intervalo de confiança reamostra **semanas**, não linhas.

> **Por quê tanto cuidado?** Em dados de séries temporais, o vazamento é fácil de criar sem querer: janelas de 28 dias precisam ser deslocadas um dia, "a mesma hora na semana anterior" só vale até 7 dias de horizonte, e a parada antecipada padrão do scikit-learn usa um sorteio aleatório interno. Todos esses pontos foram tratados.

8.7 Resultados (teste final, 2026-04 a 2026-09)

Modelo	Erro médio (MAE, pontos percentuais)	ROC-AUC	Brier	Erro de calibração
Climatologia	13,65	0,841	0,2174	0,218
Sazonal ingênuo	11,84	0,803	n/d	0,190
Gradient boosting	**10,05**	**0,917**	**0,1156**	**0,036**

- Diferença do modelo para a climatologia no MAE: **−3,6 pontos percentuais**, com intervalo de confiança de 95% de **−4,6 a −2,5** (exclui zero).
- Na validação cruzada (média das 5 dobras): MAE 9,45 (modelo), 11,54 (climatologia), 10,61 (sazonal ingênuo).

> **O que os números dizem, com honestidade:**
>
> - O modelo é **muito melhor que as linhas de base na classificação do risco e na calibração**: quando ele diz "70% de risco", acontece perto de 70% das vezes.
> - No **erro do percentual de corte** o ganho é menor, e numa dobra (2025-01 a 03) o modelo ficou **ligeiramente pior** que a climatologia (10,57 contra 10,45).
> - A precisão **quase não cai** do dia 1 ao dia 7. Isso é sinal de que o modelo é, na prática, uma "climatologia adaptativa mais calendário": sem clima, ele não enxerga o vento de amanhã.

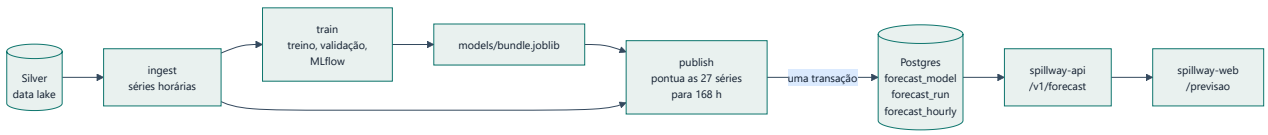
8.8 Limitações

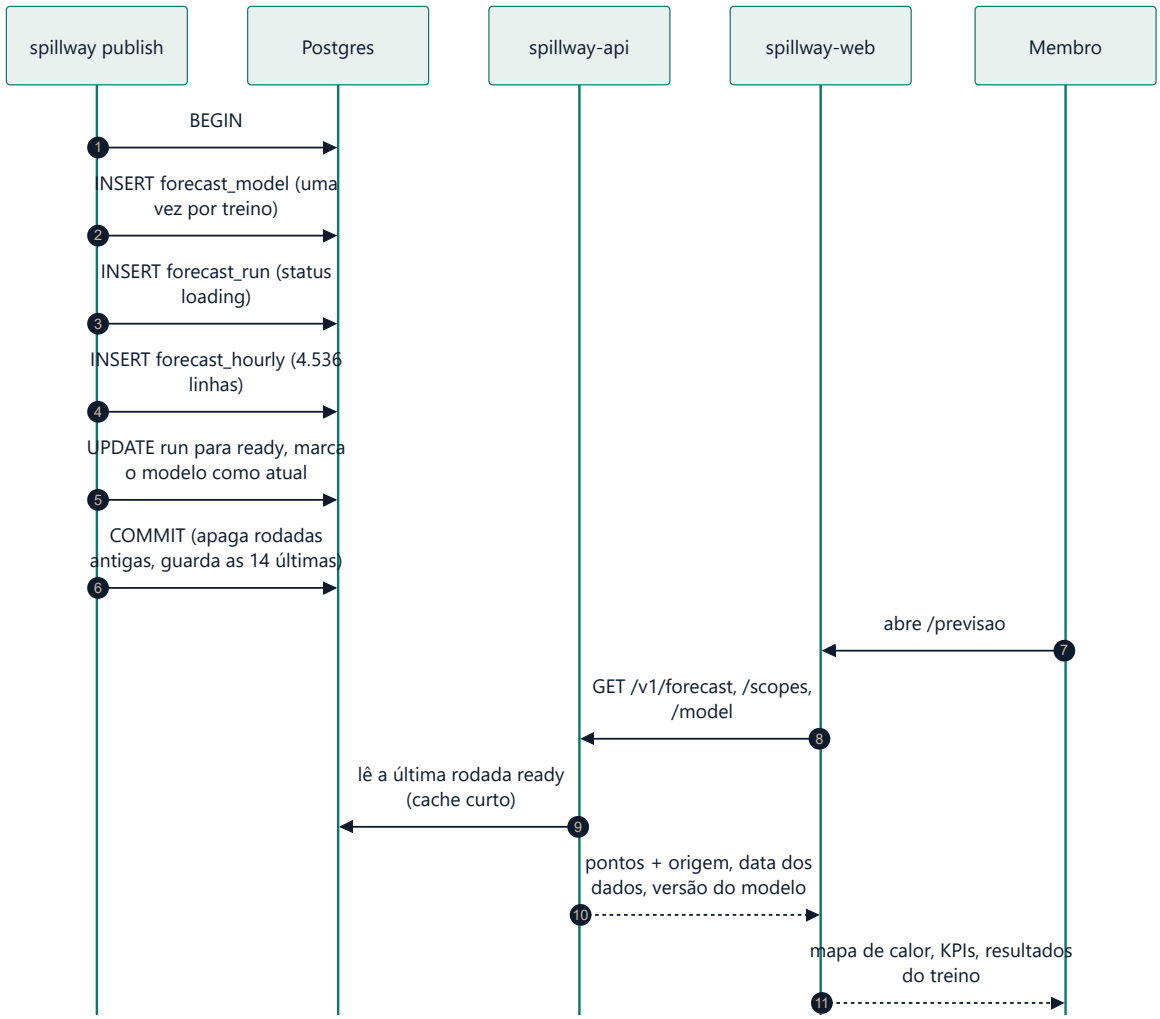
- **Sem clima.** O próximo passo natural é incluir previsão de vento e irradiação, e a carga e o preço (CMO) previstos pelo ONS.
- **Defasagem dos dados** (seção 8.2).
- Os números "apurados" do ONS são **revisados**; o último mês pode mudar.
- A solar tem só ~2,5 anos de histórico; séries por estado podem ser curtas.
- Estado ou usina **novos** não existem para o modelo (as séries são fixas no treino).

- Não serve para decisões financeiras ou operacionais sozinho, nem para alegar compensação contratual.

8.9 Como a previsão chega ao site

A decisão central (ADR 7): o Forecast **publica no banco**, a API **lê do banco**, o site chama **só a API**.





> **Por quê o banco, e não uma chamada HTTP da API ao serviço de ML?**
>
> - A API **continua funcionando** (mostrando a última previsão e a idade dela) se o job de ML estiver fora do ar.
> - O tempo de resposta não depende de pontuar um modelo no meio da requisição.
> - A imagem da API **não precisa** de scikit-learn nem de joblib.
> - Cada previsão fica **ligada à versão do modelo** que a gerou (auditoria e retorno).
> - É o mesmo padrão do pipeline: a API é dona do esquema e o produtor só escreve nas suas tabelas.

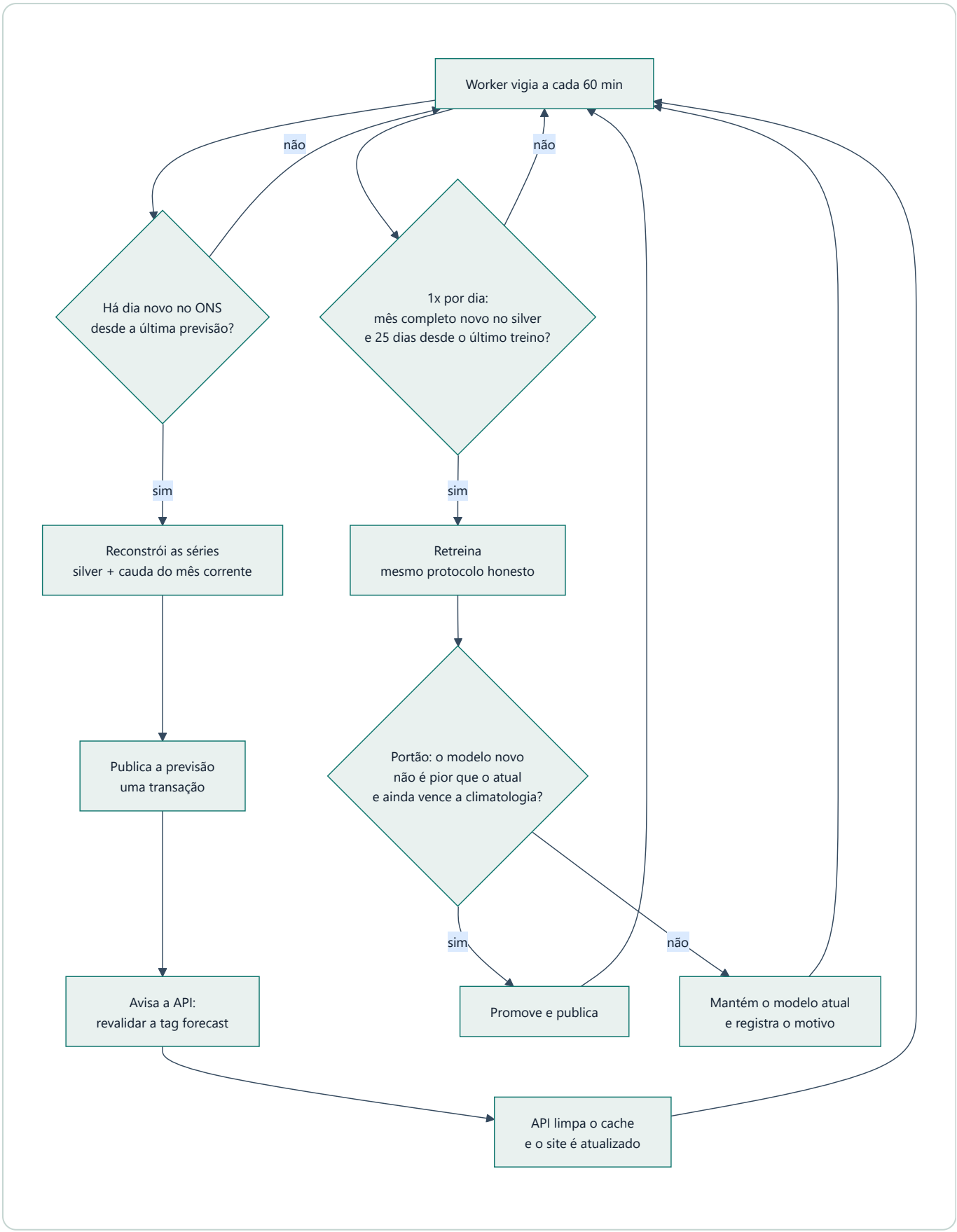
A publicação é **idempotente**: rodar de novo para o mesmo modelo e a mesma origem não faz nada. Uma rodada só fica visível quando está completa (`ready`).

8.9.1 O que a página mostra

- Mapa de calor do **risco por hora** nos próximos dias, com seletores de fonte e de região (a URL guarda a escolha, dá para compartilhar);
- Indicadores e um gráfico do corte esperado;
- Um aviso de **defasagem** ("dados do ONS até 30/09/2026");
- Os **resultados do treino em português simples**: o que é o modelo, o período dos dados, como foi validado, a tabela contra as linhas de base, o gráfico de calibração, o peso das variáveis e as limitações;
- Se o serviço estiver fora, "Previsão indisponível no momento".

8.10 Ciclo automático (em produção local desde 28/10/2026)

O ciclo abaixo está mergeado (spillway-forecast #2, spillway-api #13, spillway-web #11). O worker é um contêiner (`docker compose up -d --build forecast-worker`); sem ele, `uv run spillway refresh` faz uma atualização manual:



Pontos de proteção do retreino automático: os dois modelos (atual e candidato) são avaliados na **mesma janela de teste**; a troca é **atômica** (o modelo antigo fica guardado); existe uma **trava** para nunca rodarem dois processos ao mesmo tempo; e uma falha deixa tudo como estava.

> **Atenção:** o worker só roda se você o subir. Sem ele, a previsão fica parada até um `spillway refresh`. Para o site ser avisado na hora, a API precisa de `REVALIDATE_SECRET`; sem isso, o cache de 10 minutos do site vale.

> **Retreino medido:** o retreino completo com os dados híbridos levou cerca de 12,5 minutos e deu MAE 10,15 pp (climatologia 13,90), ROC-AUC 0,915, ECE 0,037, no mesmo nível do primeiro treino. Portão de promoção: AUC cai no máximo 0,01, MAE sobe no máximo 0,3 pp, ECE sobe no máximo 0,02 e o candidato ainda vence a climatologia com intervalo de 95%.

8.11 Como rodar

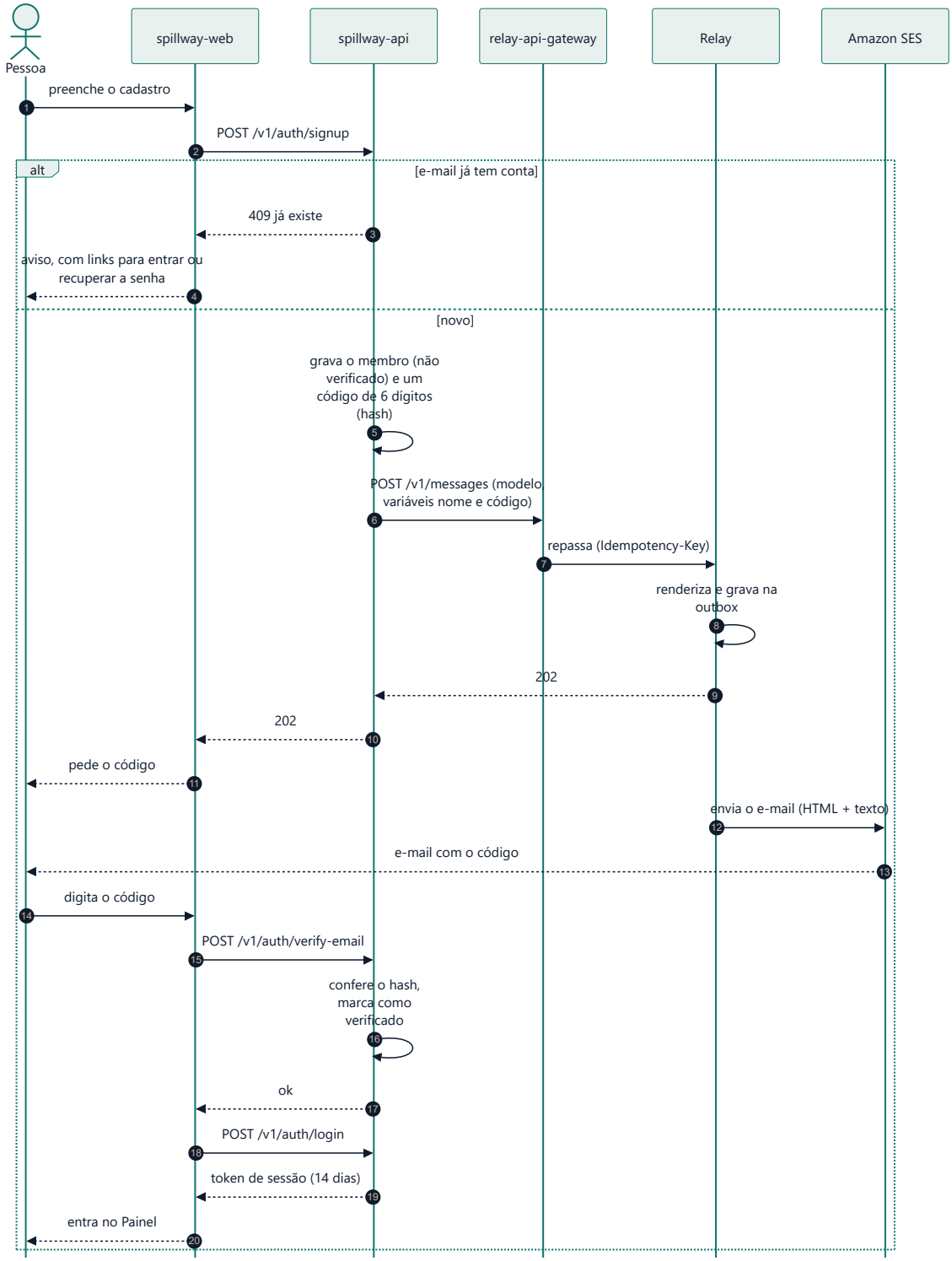
```
cd spillway-forecast
uv sync --extra lake --extra warehouse
uv run spillway ingest        # monta as séries horárias (do silver)
uv run spillway train         # ~20 min: dobras, teste final, MLflow, models/bundle.joblib
uv run spillway publish --dsn postgresql://warehouse:warehouse@localhost:5433/warehouse
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db     # navegar nas execuções
uv run pytest && uv run ruff check . && uv run mypy
```

Arquivos para consultar: `MODEL_CARD.md` (ficha do modelo: uso, dados, variáveis, validação, limites), `reports/results.json` (números completos) e `reports/reconcile.txt` (conferência das somas mensais contra o warehouse, com diferença máxima de 0,0005 MWh).

9. Fluxos de ponta a ponta

Este capítulo junta as peças. Cada fluxo mostra **quem fala com quem, em que ordem**, atravessando os projetos.

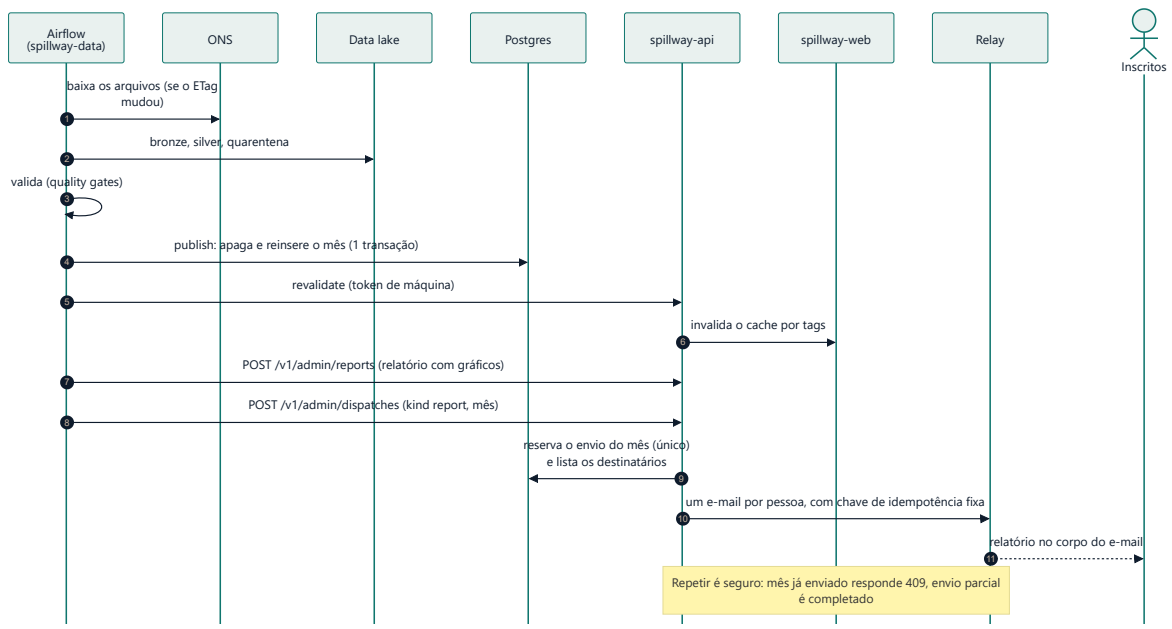
9.1 Cadastro de um membro (com código por e-mail)



Por quê código e não link? O código se digita na própria tela, funciona no celular sem abrir outro navegador e tem limite de tentativas. É mais difícil de interceptar em encaminhamentos e prévias de e-mail.

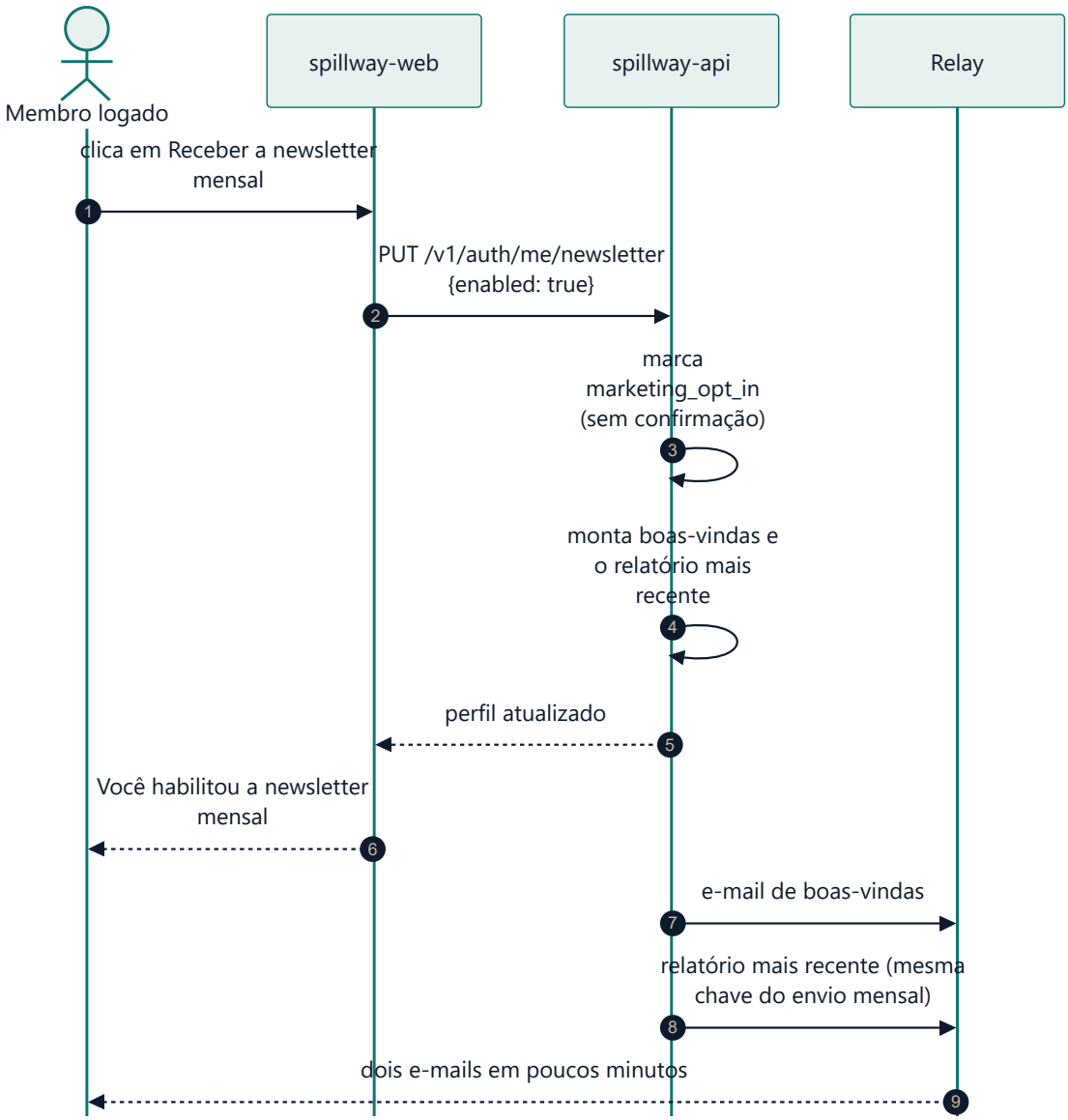
9.2 Um novo mês do ONS: do arquivo ao e-mail

É o ciclo mais importante do produto. Acontece sozinho no dia 3 de cada mês.



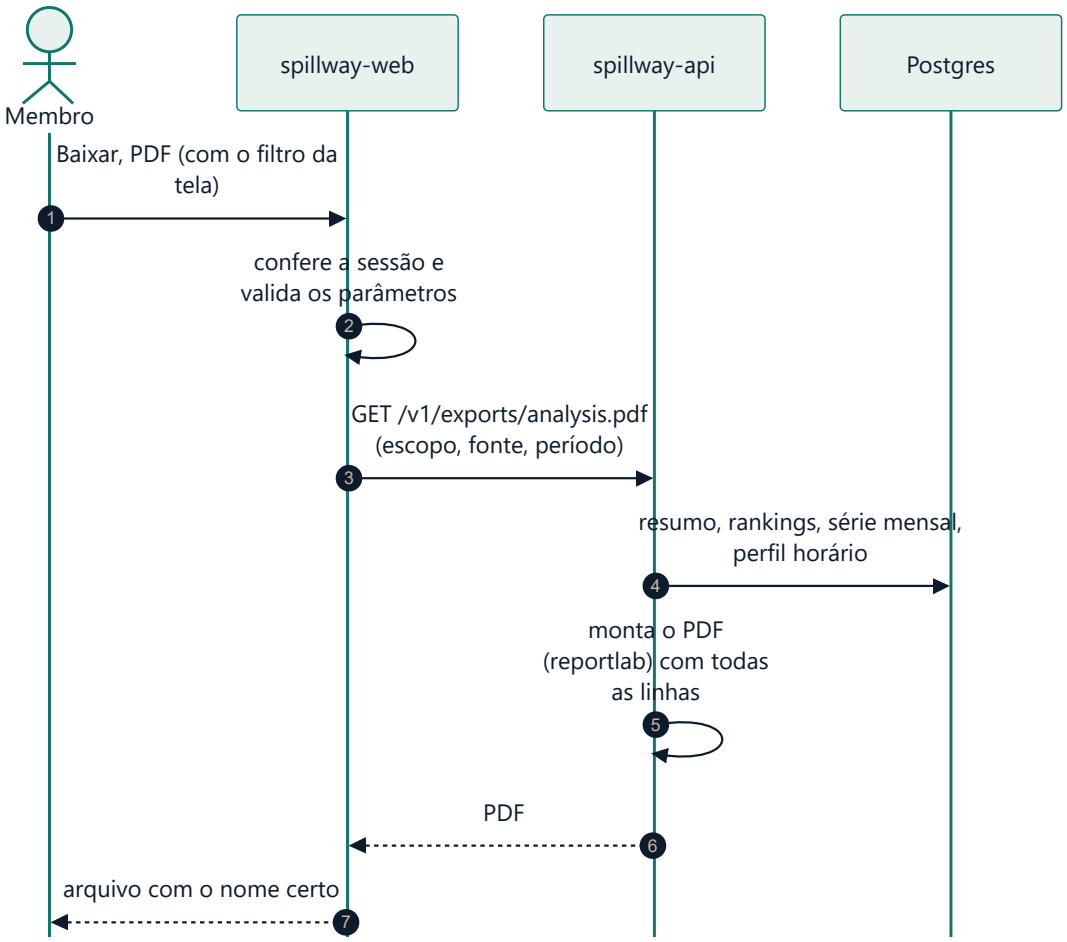
> **Por quê um envio por mês, com chave por pessoa?** Se o pipeline rodar de novo (ou o envio parar no meio), ninguém recebe duas vezes. O Relay responde "já enviei" para a mesma chave.

9.3 Ativar a newsletter na home

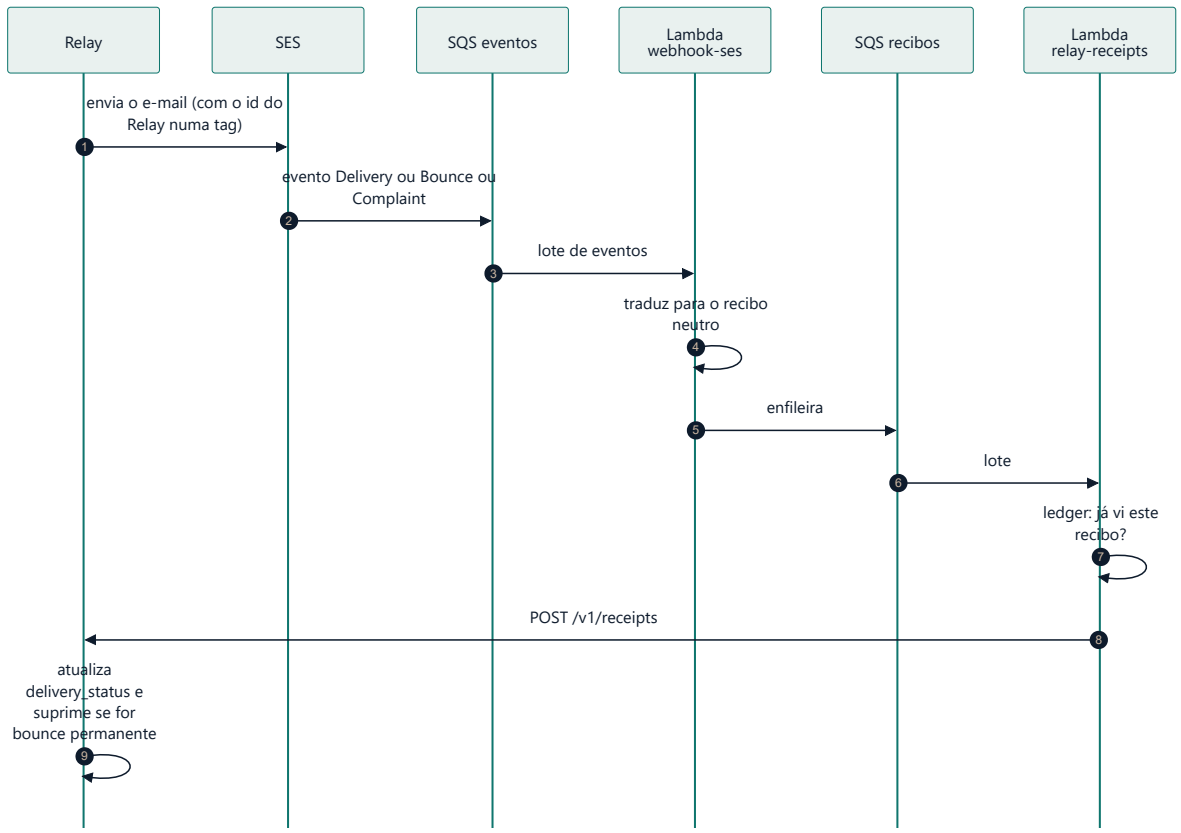


O relatório de boas-vindas é construído **exatamente** como o envio mensal o constrói, com a mesma chave de idempotência. Assim, quem ativa antes do envio do mês não recebe o relatório duas vezes.

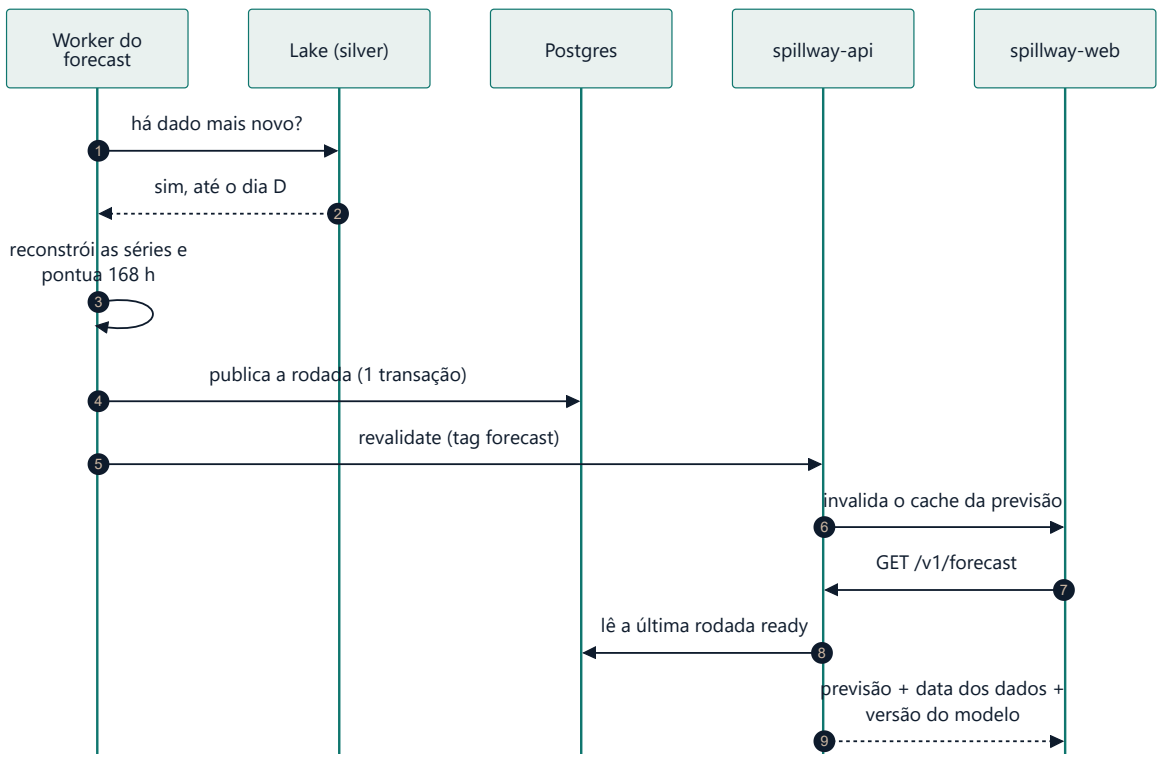
9.4 Baixar a análise em PDF



9.5 O recibo de entrega de um e-mail



9.6 Da previsão ao site



> **Atenção:** o worker precisa estar rodando (capítulo 8.10) e a API precisa de `REVALIDATE_SECRET` para o site atualizar na hora.

9.7 Esqueci a senha

Segue o mesmo desenho do cadastro: a API gera um código (modelo "redefinir senha"), o Relay entrega, a pessoa digita o código com a nova senha, e a API troca a senha e **encerra as sessões antigas**.

9.8 Resumo: onde cada decisão aparece

Fluxo	Decisões em ação
Cadastro	Código por e-mail; Relay com outbox e idempotência; modelos de e-mail do próprio produto
Novo mês	Carga idempotente com reprocessamento; invalidação de cache por evento; envio único por mês
Newsletter	Mesma chave do envio mensal; sem confirmação para quem já verificou o e-mail
Recibos	Fila entre as Lambdas; recibo neutro; status que só avança
Previsão	Publicação pelo banco; API dona do esquema

10. Operação: como subir, onde está cada coisa, o que fazer quando algo quebra

10.1 O mapa de portas (ambiente local)

Serviço	Porta	O que é
spillway-web (dev)	**3002**	O site
spillway-api	**8000**	API do Spillway
Airflow	**8081**	Interface do pipeline (login `airflow` / `airflow` localmente)
Postgres do warehouse	**5433**	Banco do Spillway (`warehouse` / `warehouse`)
Data lake (SeaweedFS)	**8333** (S3) e **8888** (navegador de arquivos)	Bronze, silver, gold
Relay API	**3000**	Mensagens
Relay publisher	3001	Métricas do publisher
relay-api-gateway	**8080**	Porta de entrada do Relay
Postgres do Relay	5432	Banco do Relay
Redpanda (Kafka)	19092	Fila
Prometheus / Grafana	9090 / 3030	Métricas
Provedores simulados	8091 a 8096	SMS, e-mail, WhatsApp, push de mentira
Serviço de depuração do forecast	8100	Opcional, fora do caminho de produção

> **Por quê o site na 3002?** O Relay já usa a 3000 e a 3001. O Airflow ficou na 8081 (e não na 8080) porque o gateway do Relay ocupa a 8080.

10.2 Subindo tudo, na ordem

```
# 1) Relay (mensagens)
cd relay
docker compose up -d --build api publisher worker postgres redis redpanda
# com PROVIDERS=providers.real.json para usar o SES de verdade

# 2) Gateway do Relay
cd ../relay-api-gateway
docker compose up -d --build

# 3) Pipeline e banco do Spillway
cd ../energia-data
docker compose up -d        # Airflow, lake, warehouse (use o override de porta se a 8080 estiver ocupad

# 4) Migrações do banco (cria as tabelas)
cd ../energia-api
DATABASE_URL=postgresql+psycopg://warehouse:warehouse@localhost:5433/warehouse uv run alembic upgrade he

# 5) API do Spillway
ENVIRONMENT=local AUTH_DISABLED=true DATABASE_URL=postgresql+psycopg://warehouse:warehouse@localhost:543
WEB_URL=http://localhost:3002 EMAIL_BACKEND=gateway GATEWAY_URL=http://localhost:8080 GATEWAY_API_KEY=..
uv run uvicorn app.main:app --port 8000

# 6) Site
cd ../energia-web
API_URL=http://127.0.0.1:8000 pnpm exec next dev -p 3002
```

Dica: para a velocidade real do site, use o build de produção em vez do `next dev`:
`NEXT_DIST_DIR=.next-prod pnpm build && NEXT_DIST_DIR=.next-prod pnpm exec next start -p 3002`.

10.3 Dados: backfill e relatórios

```
# Backfill (carrega muitos meses) pelo Airflow
docker exec energia-data-airflow-scheduler-1 airflow dags unpause ons_curtailment_pipeline
docker exec energia-data-airflow-scheduler-1 airflow dags trigger ons_curtailment_pipeline \
  --conf '{"first_month": "2021-10", "last_month": "2026-09"}'

# Gerar e publicar os relatórios de todos os meses
docker exec energia-data-airflow-scheduler-1 python -m energy_pipeline.report \
  --from 2021-10 --to 2026-09 --publish --submit
```

10.4 Envios de e-mail

Quero	Comando ou rota
Enviar o relatório do mês mais recente	`uv run python -m app.dispatch report` (na pasta da API) ou `POST /v1/admin/dispatches`
Terminar um envio que parou	acrescentar `--retry`
Reenviar um mês já enviado	apagar a linha do mês em `email_dispatch` (e saber que o Relay devolve "já enviado" para as mesmas chaves)
Mandar um e-mail genérico, sem modelo	`POST /v1/messages` no gateway, só com `channel`, `to`, `subject` e `body`

10.5 Testes e verificações por repositório

Repositório	Comando
spillway-api	`uv run ruff check . && uv run mypy && uv run pytest -q`
spillway-web	`pnpm exec tsc --noEmit && pnpm exec eslint src && pnpm exec vitest run`
spillway-data	`uv run ruff check . && uv run pytest -q`
relay	`npm test`
relay-api-gateway	`pnpm test`
relay-receipts	`uv run ruff check . && uv run mypy && uv run pytest -q`
spillway-forecast	`uv run pytest && uv run ruff check . && uv run mypy`
relay-terraform e spillway-infra	`terraform fmt -check && terraform validate`

Todos os repositórios têm **CI no GitHub Actions** com lint, tipos, testes, varredura de segredos e auditoria de dependências (e, nos de imagem Docker, varredura de vulnerabilidades com Trivy).

10.6 Onde ficam segredos e configurações

O quê	Onde
Chaves do Relay e do gateway	Arquivos `.env` **fora do Git** em cada repositório
Chave de acesso do SES local	`.env` do Relay (saída do Terraform)
Estado do Terraform do `relay-terraform`	Arquivo local, **fora do Git** (contém o segredo da chave IAM)

O quê	Onde
Segredos de produção	AWS Secrets Manager e SSM (previstos no `spillway-infra`)

> **Atenção:** o estado local do Terraform do `relay-terraform` guarda o segredo da chave de acesso do usuário IAM. Mantenha esse arquivo fora do Git (já está) e, para uso em equipe, migre para um backend S3 criptografado.

10.7 Lista de problemas que já aconteceram (e a solução)

Sintoma	Causa	O que fazer
Página mostra "Dados indisponíveis" ou lista antiga depois de carregar dados	Cache de `fetch` do Next persistiu	Parar o site, apagar `energia-web/.next/cache/fetch-cache`, subir de novo
Rota nova da API responde 404	O processo da API é antigo	Reiniciar o processo da API
Erro 500 ao cadastrar depois de mudar templates	Processo da API antigo com arquivo de template removido	Reiniciar a API
`RELAY_URL must start with http://` ao testar a Lambda `relay-receipts`	A Lambda está desligada de propósito (Relay não é público)	Esperado. Ligar exige URL pública e `terraform apply`
Recursos "não aparecem" na AWS	Console aberto em outra região	Trocar para **sa-east-1 (São Paulo)**
Docker Desktop parou durante um build grande	Falta de memória do Docker	Reiniciar o Docker Desktop e subir de novo (os volumes permanecem)
Porta 8080 em uso ao subir o Airflow	O gateway do Relay usa a 8080	Usar o override de porta (Airflow na 8081)
Alarme `relay-dlq-ses_events` em ALARM	16 mensagens antigas na DLQ, restos de testes	Limpar a fila (`purge-queue`) quando quiser
Previsão com datas antigas	Último dado do ONS no silver é 30/09	Já corrigido pela ingestão híbrida (capítulo 8.10): rode `uv run spillway refresh` ou suba o worker
`terraform apply` não mostra mudanças para recursos que "sumiram"	Os recursos existem, em outra região	Conferir a região

10.8 Cuidados antes de alterar algo

- **Banco compartilhado (warehouse):** migrações e cargas escrevem nele. Rodar `spillway publish`, o backfill ou o `alembic upgrade` altera dados reais do ambiente local.
- **Nomes implantados** (escopo do Cognito, ECR, usuários do banco, caminhos na EC2): não renomear sem planejar a migração.
- **SES:** a conta está em sandbox; um envio de teste só vai para endereços verificados.
- **Disco:** o backfill e as imagens do Docker ocupam espaço; se faltar, `docker system df` mostra o que é recuperável.

11. As decisões de arquitetura, em linguagem simples

Uma decisão de arquitetura é uma escolha com **custo**. Aqui cada uma está no mesmo formato: qual era o problema, o que foi escolhido, o que foi descartado e que custo se aceitou. Os textos completos estão nos **ADRs** (*Architecture Decision Records*) dentro dos repositórios.

11.1 Spillway (ADRs do `spillway-api`)

#	Decisão	Problema	Alternativa descartada	Custo aceito
1	**Quatro repositórios com um contrato escrito; a API é dona do esquema**	Quatro partes que evoluem juntas podem discordar sobre tabelas e rotas	Um repositório único (monorepo), ou cada parte criando suas tabelas	Mudar algo compartilhado começa por um documento e passa por mais de um repositório
2	**O número oficial do ONS vale; distribui-se para as usinas**	A estimativa por usina não fecha com o total oficial	Usar só a estimativa das usinas	A distribuição por usina é uma regra de modelagem (explicada na metodologia)
3	**Cargas mensais idempotentes, reprocessando os últimos 3 meses**	O ONS revisa dados antigos; rodar de novo não pode duplicar	Carregar só o mês novo e nunca revisar	Mais processamento por rodada
4	**Invalidação do cache disparada por evento (do pipeline para o site)**	Esperar uma hora para ver dados novos é ruim, mas consultar sempre é caro	Só cache por tempo; ou nenhum cache	Uma cadeia de chamadas (pipeline, API, site) que precisa de segredos
5	**Sem NAT gateway**	Custa uns US$ 40 por mês parado	NAT gateway em todas as redes	Tarefas em sub-redes públicas, com toda a entrada fechada
6	**Deploy sem chaves, por OIDC, com ambiente de produção aprovado por pessoa**	Chaves da AWS no GitHub vazam	Chaves de acesso guardadas como segredo	Configuração inicial de papéis e confiança
7	**A previsão é publicada pelo banco, não chamada por HTTP**	A API não pode depender de um serviço de ML estar no ar	Proxy HTTP da API para o serviço de ML	Uma camada de tabelas e um job de publicação

11.2 Relay (ADRs do `relay`)

#	Decisão	Problema	Alternativa descartada	Custo aceito
1	**Outbox transacional com um publicador líder**	Escrever no banco e no Kafka separadamente perde ou duplica mensagens	Publicar no Kafka direto da API	Um processo a mais e um pequeno atraso até o Kafka
2	**Partição por destinatário, com concorrência por chave**	Manter a ordem por pessoa sem serializar tudo	Um consumidor único, ou sem ordem	O paralelismo máximo é o número de partições
3	**Pelo-menos-uma-vez com idempotência em cada salto**	"Exatamente uma vez" não existe de ponta a ponta com provedores externos	Tentar garantir exatamente uma vez	Um duplicado raro quando o provedor não tem chave de idempotência
4	**Filas de retry em camadas e fila de mortos**	Esperar sem prender o consumidor nem usar um cronômetro por mensagem	Dormir no consumidor ou agendar por mensagem	Atrasos fixos (5 s, 30 s, 5 min)
5	**Troca de provedor com circuit breaker por processo**	Um provedor morto custaria um timeout por mensagem	Estado de saúde compartilhado em um serviço	Cada servidor aprende a falha por conta própria
6	**Limite de taxa para toda a frota num balde no Redis**	O contrato é por conta, não por servidor	Limite por processo	Redis no caminho quente (sem ele, os workers param)
7	**Recibos de provedores, status que só avança e lista de supressão**	O que o provedor diz depois (bounce, spam) precisa voltar, e fora de ordem	Ignorar os eventos	Duas dimensões de status (`status` e `delivery_status`)
8	**O produto manda o HTML; o Relay preenche as variáveis**	O Relay serve vários produtos e não deve virar catálogo de modelos	Guardar modelos dentro do Relay	O Relay não tem condicionais nem laços: o produto renderiza se precisar

11.3 Relay-receipts

Decisão	Problema	Escolha	Custo aceito
SQS entre as duas Lambdas, não API Gateway	O Relay pode estar fora do ar; picos de eventos	Fila (guarda por 14 dias, tentativas, DLQ, sem endpoint público)	Entrega pelo-menos-uma-vez, resolvida com ledger e idempotência

Decisão	Problema	Escolha	Custo aceito
Recibo neutro de um único formato	O Relay não deve conhecer o vocabulário de cada provedor	Um tradutor por provedor (hoje o `webhook-ses`) e um webhook geral	Escrever um tradutor novo por provedor

11.4 Forecast

Decisão	Problema	Escolha	Custo aceito
Gradient boosting global, com duas linhas de base	Prever o risco por hora com histórico limitado e sem clima	Árvores (HistGradientBoosting), um modelo para 27 séries	Sem clima, o modelo é "histórico mais calendário"
Validação temporal com folga e teste final intocado	Evitar vazamento de futuro	5 dobras crescentes, embargo de 7 dias, teste de vazamento	Validação mais pesada (cerca de 20 minutos de treino)
Publicar no banco em uma transação	A previsão nunca pode aparecer pela metade	Rodada `loading` e depois `ready`	Mais tabelas
Retreino automático com portão	Um modelo novo pode ser pior que o atual	Promover só se não piorar e ainda vencer a climatologia	Lógica extra de comparação

11.5 Decisões de produto e de experiência

Decisão	Por quê
Só a página inicial, a metodologia e a privacidade são abertas	O site é um produto para membros e a base alimenta o e-mail mensal
Menu por pergunta (Painel, Explorar, Análises, Previsão, Relatórios)	Menos itens e sem redundância entre relatórios, painel e análises
Comparar é uma ação dentro de Estados e Usinas	Compara-se a partir de algo que se está olhando
Cadastro com e-mail existente avisa claramente	Clareza para o usuário valia mais que esconder se o e-mail existe
Relatório inteiro no corpo do e-mail (sem anexo por enquanto)	Mais simples e mais lido; o anexo PDF fica para depois
Dois downloads: PDF ou dados completos	Um PDF completo e um conjunto de dados, em vez de muitos arquivos soltos
Newsletter com um clique para quem está logado	O e-mail já foi verificado; confirmar de novo seria atrito sem ganho

> **Em resumo:** quase todas as decisões trocam um pouco de simplicidade por **segurança contra falha** (nada se perde, nada duplica, cada serviço pode cair sozinho). Em um produto que depende de dados oficiais e de e-mail, é a troca certa.

11.6 Novas fontes do ONS

Decisão	Por quê	Custo aceito
Um pipeline por assunto (sistema, hidrologia, rede)	Isolar falhas e horários; o curtailment não muda	Mais DAGs para acompanhar
Tabela própria para a frota (`power_plant`) em vez de ampliar `plant`	`plant` é eólica e solar do produto de curtailment	Duas tabelas de usinas, com o mesmo id
Cadastro da rede com histórico (`valid_from` , `valid_to`)	O mapa da rede vai mostrar a expansão	Uma lógica de comparação por hash no publish
Gates que dizem a verdade dos dados (reservatório negativo, `SIN` no balanço, ruído solar)	Bloquear só o que é erro, avisar o que é incomum	É preciso olhar os avisos de vez em quando
Backfill de hidrologia só de 2015 em diante	Os arquivos antigos são grandes e raros de usar	O histórico anterior fica para uma execução à parte

12. Estado atual, pendências e riscos

> **Honestidade:** este capítulo descreve o estado conhecido ao final da escrita desta documentação. Atualizado em 29/10/2026, depois do merge do ciclo automático do forecast.

12.1 O que está pronto e funcionando (no seu computador)

Área	Estado
Pipeline de dados	Backfill completo de **2021-10 a 2026-09** (60 meses; eólica desde 2021-10, solar desde 2024-04). Relatórios dos 60 meses gerados, com gráficos.
Site	Menu por pergunta, área de membros, painel, estados, usinas, comparar, análises com download em PDF, Excel e CSV, relatórios com PDF, previsão com barra de progresso, tema claro e escuro.
API	Contas com código por e-mail, exportações, relatórios, disparo mensal, newsletter com um clique, boas-vindas e relatório por e-mail, previsão publicada pelo banco.
Relay	Plataforma completa com testes e carga; templates e variáveis; recibos de entrega; **e-mails reais entregues pelo SES** (verificado nos seus envios).
Forecast	Treinado com **todos os anos**, validado com protocolo temporal, publicado no banco local, com ficha do modelo na tela.
Documentação	Esta, em Markdown, HTML e PDF.

12.2 O que foi concluído por último e o que está aberto

- **Mergeado:** ciclo automático da previsão (ingestão híbrida, worker que vigia a cada hora, tag `forecast`, aviso de defasagem, janela que parte de hoje), spillway-forecast #2, spillway-api #13 e spillway-web #11. **Falta subir o worker** para ele rodar sozinho.
- **Retreino mensal automático** com portão de qualidade e troca atômica (mergeado, ativo quando o worker roda).
- **spillway-web #12 (aberto):** gráfico de confiabilidade legível e conclusão com cuidados.
- **spillway-data #3:** renomeação das referências para `spillway-*` (CI vermelho por vulnerabilidades da imagem do Airflow que já existiam).

12.3 O que ainda não existe

Item	Situação
Spillway em produção na AWS	O Terraform está escrito e validado, mas **nada foi aplicado**. O roteiro de primeira implantação está no `spillway-infra`.

Item	Situação
Renomear as pastas locais `energia-*` para `spillway-*`	Pendente: exige parar API, web e Airflow e fechar editores. O `docker-compose.yml` do `spillway-data` na `main` ainda aponta para um caminho antigo (o PR de renomeação corrige).
SES fora do sandbox	A conta só envia para endereços verificados.
Relay com URL pública	Necessário para ligar a Lambda `relay-receipts`.
Ciclo automático em produção	Hoje tudo roda localmente; em produção, o worker do forecast viraria uma tarefa do ECS.
Comparativo de modelos	Só foram medidas duas linhas de base; falta comparar com LightGBM, Prophet e redes neurais, se for importante.
Anexo PDF no e-mail	Planejado para depois ("primeiro no corpo").
Clima na previsão	Próximo grande ganho possível.

12.4 Riscos e pontos de atenção

Risco	Impacto	Mitigação
Dado do ONS com defasagem de semanas	A previsão "não é de hoje"	Ingestão híbrida da cauda do mês corrente (mergeada) e aviso claro na página
Retreino automático piorar o modelo	Previsão pior sem ninguém ver	Portão de promoção e modelo anterior guardado
Nomes antigos em recursos implantados	Confusão e quebra se alguém renomear sem migrar	Documentado; não renomear sem planejar
Confiança do OIDC aponta para os nomes novos	O deploy por GitHub Actions falha até aplicar o Terraform	`terraform apply` no `spillway-infra` na primeira implantação
Estado local do Terraform do `relay-terraform`	Contém o segredo da chave IAM	Fora do Git; migrar para S3 criptografado em equipe
Cache local de `fetch` do Next	Mostra dados antigos depois de carga	Apagar `.next/cache/fetch-cache`; em produção a invalidação é automática
SES em sandbox	Envio só para você	Pedir a saída do sandbox antes de abrir o produto
Alarme da DLQ do SES em ALARM	Ruído	Limpar a fila de restos de teste
Vulnerabilidades da imagem do Airflow	CI vermelho no scan	Atualizar a imagem base e os pacotes quando houver correção

12.5 Próximos passos sugeridos

1. **Revisar e mergear** os PRs do ciclo automático do forecast quando terminarem.
2. **Colocar o Spillway no ar**: seguir o roteiro do `spillway-infra`, começando pelo estado do Terraform e pelas variáveis.
3. **Abrir o e-mail**: tirar o SES do sandbox, e decidir se o Relay terá URL pública.
4. **Observar o primeiro ciclo mensal real** (dia 3): conferir o relatório publicado, o e-mail e a previsão.
5. **Evoluir o modelo**: incluir clima e comparar com outras famílias de modelos.
6. **Anexar o PDF** do relatório ao e-mail.

12.6 Resumo em números

Repositórios	10 (9 com GitHub)
Meses de dados	60 (2021-10 a 2026-09)
Séries do forecast	27
Testes	API ≈ 240, site ≈ 160, Relay 86, relay-receipts 74, data 91
Capacidade medida do Relay (1 API)	≈ 290 msg/s aceitas; entrega sustentada de 250 msg/s
Custo mensal estimado na AWS	US$ 170 a 270

12.7 Novas fontes do ONS (fases 0 a 4)

Fase	O que entrou	Situação
0	Arquivos anuais até um mês e fallback parquet/csv	Mergeado
1	Painel do SIN: balanço, geração por fonte e por usina	Mergeado
2	Intercâmbio e despacho térmico	Mergeado
3	Água e energia (reservatórios, EAR, ENA, energia vertida)	Mergeado
4	Cadastro da rede com histórico	Mergeado

Ainda **não existe**: as telas do portal (Painel do SIN, Água e energia, Mapa da rede), que só consomem estas rotas; as coordenadas das linhas (virão da ANEEL/SIGEL); e o backfill da hidrologia antes de 2015. Nada disso foi aplicado na AWS: o Terraform do `spillway-infra` já tem as permissões novas, mas a implantação de produção continua pendente.

13. Glossário

Setor elétrico

Termo	Significado
Curtailment	Energia eólica ou solar que **poderia** ter sido gerada, mas foi reduzida por ordem do operador.
ONS	Operador Nacional do Sistema Elétrico: quem coordena a operação da rede e publica os dados abertos.
CCEE	Câmara de Comercialização de Energia Elétrica: forma os preços usados para valorar a energia.
SIN	Sistema Interligado Nacional (o Brasil inteiro, para este produto).
Subsistema	N, NE, S e SE (Sudeste/Centro-Oeste).
GNRa	Geração não realizada apurada: a energia cortada oficial.
REL	Corte por razão de **confiabilidade elétrica**.
CNF	Corte por **indisponibilidade externa** (equipamento fora da usina).
ENE	Corte por **razão energética** (mais oferta que consumo).
CMO	Custo Marginal de Operação (usado para valorar a energia cortada).
PLD	Preço de Liquidação das Diferenças (tem piso e teto anuais).
Compensável	A parte do corte (REL e CNF) que pode dar direito a ressarcimento.
MWh / GWh	Megawatt-hora e gigawatt-hora (1 GWh = 1.000 MWh).

Dados e engenharia

Termo	Significado
Pipeline	Sequência automática de passos que leva um dado da origem até o destino.
Airflow	Agendador e orquestrador de pipelines.
DAG	O desenho do pipeline: tarefas e a ordem entre elas.
Bronze, silver, gold	Camadas do data lake: original, limpo e validado, agregado para consumo.
Data lake	Armazenamento de arquivos (aqui, S3 ou SeaweedFS).
Backfill	Carregar muitos meses antigos de uma vez.
ETag	Identificador da versão de um arquivo: se não mudou, não precisa baixar de novo.
Idempotente	Que dá o mesmo resultado se for repetido; seguro de rodar de novo.

Termo	Significado
Migração	Mudança versionada no esquema do banco (Alembic).
ADR	Registro curto de uma decisão de arquitetura e seu motivo.
CI	Verificações automáticas em cada mudança de código (GitHub Actions).
OIDC	Forma de o GitHub provar sua identidade à AWS sem chaves fixas.
Terraform	Descreve infraestrutura em código.

Relay e mensageria

Termo	Significado
Outbox	Tabela que guarda a mensagem junto com o dado, na mesma transação, para publicá-la depois.
Kafka (Redpanda)	Fila de mensagens distribuída e durável.
Partição	Divisão de um tópico do Kafka; a mesma chave cai sempre na mesma partição.
Offset	Posição de leitura numa partição.
DLQ	*Dead-letter queue*: fila de mensagens que não puderam ser processadas.
Circuit breaker	Disjuntor que para de chamar um provedor com falhas por um tempo.
Failover	Passar para outro provedor quando o primeiro falha.
Token bucket	Balde de fichas: limita quantas chamadas por segundo se pode fazer.
Idempotency-Key	Cabeçalho que identifica uma requisição, para repeti-la sem duplicar.
SES	Amazon Simple Email Service, o provedor de e-mail.
SNS / SQS	Notificação e fila da AWS.
Lambda	Função que roda na AWS sob demanda.
Bounce	E-mail devolvido (endereço inexistente, caixa cheia).
Complaint	A pessoa marcou o e-mail como spam.
Supressão	Lista de endereços para os quais não se envia mais.
Sandbox (SES)	Modo inicial em que só se envia a endereços verificados.

Previsão e aprendizado de máquina

Termo	Significado
Série temporal	Valores medidos ao longo do tempo (aqui, por hora).

Termo	Significado
Climatologia	O valor típico para aquela época e hora, aprendido do histórico. Usada como linha de base.
Gradient boosting	Combina muitas árvores de decisão pequenas, cada uma corrigindo as anteriores.
Variável (feature)	Informação que o modelo recebe para decidir.
Vazamento (leakage)	O modelo "ver" informação do futuro no treino, o que infla os resultados.
Validação temporal	Testar sempre em período posterior ao do treino.
Embargo	Folga entre o fim do treino e o início do teste.
MAE	Erro absoluto médio (aqui, em pontos percentuais).
ROC-AUC	Quão bem o modelo separa horas de risco das sem risco (0,5 é sorte, 1 é perfeito).
Brier	Erro quadrático das probabilidades (menor é melhor).
Calibração (ECE)	Se "70% de risco" acontece mesmo ~70% das vezes (menor erro é melhor).
Bootstrap	Reamostragem para estimar a incerteza de uma diferença.
MLflow	Registro das execuções e versões dos modelos.
Retreino	Treinar de novo com os dados mais novos.
Defasagem (stale)	Idade do dado mais recente usado.